

POLITEHNICA UNIVERSITY OF BUCHAREST

SOFTWARE ENGINEERING

SKILLHUB

Software Design Document

Team:

FINICHIU Eduard-Adelin 1241EB

JERCAU Hadasa-Stefana 1241EB

Coordinator:

Prof. Dr. Ing Nicolae Goga

Date created:

Sunday, November 16, 2025

Delivery Report

(will be delivered along with the project)

Name	Group	Project implementation [%, reason]	Signature
------	-------	------------------------------------	-----------

FINICHIU Eduard Adelin	1241 EB	50%(diagrams, review,information)	FEA
------------------------	---------	-----------------------------------	-----

JERCAU Hadasa Stefana	1241 EB	50%(chapter 3,4, Apendinces)	JHS
-----------------------	---------	------------------------------	-----

Delivery date:

17.11.2025

Table of Contents

DELIVERY REPORT.....	2
TABLE OF CONTENTS	3
SYSTEM DESIGN.....	4
Document Change History	4
1. Introduction.....	4
1.1. Purpose	4
1.2. Target Public.....	4
1.3. Definitions, Acronyms and Abbreviations.....	4
1.4. Document Structure	5
2. References	6
3. Decomposition Description.....	7
3.1. Modules Description	7
3.2. Description of Concurrent Processes.....	8
3.3. Description of Data Modules	10
4. Dependency Description	12
4.1. Dependencies among modules	12
4.2. Dependencies among processes.....	13
4.3. Dependencies among data modules	14
5. Interface Description	16
5.1. Module Interfaces	16
5.2. Processes Interfaces	Error! Bookmark not defined.
6. Detailed Design	22
6.1. Modules detailed design	22
6.2. Data Modules Detailed Design	28
APPENDICES	32
A1. Use cases diagrams	32
A2. Class diagrams	32
A3. Sequence diagrams	32
A4. Document evolution	33
A5. Conclusions on activity.....	39

System Design

According to the IEEE STD-1016-1998, *IEEE Recommended Practice for Software Design Descriptions*.

Document Change History

Depending on the chosen type of development, retention of all changes made to this document may be useful. For example, for a project using the *Waterfall w/ milestones* methodology, all the changes made between two project evaluation moments (*milestones*) will be retained. A chronological sorting of the list of changes is indicated.

Version	Date	The author/authors of the change	Details of changes
1.00	28.03.2004	Prof Dr Nicolae Goga	Basic version (template)
1.10	10.11.2025	FINICHIU Eduard-Adelin	Completed 1, 2
1.20	13.11.2025	JERCAU Hadasa-Stefana	Completed 3,4
1.30	14.11.2025	FINICHIU Eduard-Adelin	Completed 5,6
1.40	16.11.2025	JERCAU Hadasa-Stefana	Completed Apendices
1.50	17.11.2025	FINICHIU Eduard-Adelin	Review of Document + Sync info with SRS

1. Introduction

1.1. Purpose

The purpose of this document is to present the comprehensive software design of the SkillHub platform, a coursework project developed for the Software Engineering discipline. This document describes the system's structure, its architectural components, the modules and processes it contains, and the interactions between them. Its goal is to provide a clear and detailed perspective on how the system implements the functional and non-functional requirements previously defined in the Software Requirements Specification (SRS).

The Software Design Document (SDD) ensures a common understanding of the system among all team members by detailing the system decomposition, module responsibilities, dependencies, interfaces, and data structures. It serves as a technical guide for the development, integration, and maintenance of the SkillHub platform.

1.2. Target Public

This document is intended for all members involved in the development and evaluation of the SkillHub platform. The target audience includes:

- Software designers and system architects, who require a detailed overview of the system's structure and module interactions.
- Programmers, who use the module descriptions, interfaces, and data structures to correctly implement and integrate the system components.
- Project managers, who rely on this document to understand the system architecture, track design decisions, and coordinate development activities.
- Academic evaluators (course instructors), who analyze the system's design quality, consistency, and alignment with the Software Requirements Specification (SRS).

The document is *not* intended for end-users or clients, as it focuses strictly on the technical design aspects of the SkillHub platform.

1.3. Definitions, Acronyms and Abbreviations

This section lists the main terms and abbreviations used throughout this document to ensure clarity and consistency. The definitions are provided so that the document can be understood by readers with varying technical backgrounds.

SkillHub – The software platform developed in this project, designed to connect clients with freelancers through job postings, proposals, and notifications.

SRS (Software Requirements Specification) – The document that defines the functional and non-functional requirements of the system.

SDD (Software Design Document) – This document, which details the system's architecture, modules, interfaces, and data structures.

Client – A user who creates job postings and selects freelancers based on proposals.

Freelancer – A user who browses posted jobs and submits proposals (bids) for available work.

Module – A functional component of the system that encapsulates a specific set of responsibilities.

Process – A sequence of operations executed by the system, possibly running concurrently (e.g., sending notifications).

Data Module – A component responsible for storing, retrieving, and managing structured data used by the system.

DAO (Data Access Object) – A design pattern used to abstract and encapsulate all access to the database.

MVC (Model–View–Controller) – A software architectural pattern used to separate concerns in the system's structure, where applicable.

API (Application Programming Interface) – A defined interface for communication between modules or external systems.

Notification – A system-generated message informing a user of an event (e.g., receiving a proposal, proposal acceptance).

Proposal/Bid – A structured offer submitted by a freelancer, containing pricing and a description for a specific job.

FR – Functional Requirement

HTTP – HyperText Transfer Protocol

NFR – Non-Functional Requirement

SRS – Software Requirements Specification

1.4. Document Structure

This document is organized into several chapters, each describing a specific aspect of the system's design:

- Chapter 1 – Introduction
Provides the purpose, target audience, terminology, and overall structure of the Software Design Document.
- Chapter 2 – References
Lists the standards, documents, and external sources used as the basis for the system's design.
- Chapter 3 – Decomposition Description
Presents the system's architectural breakdown into modules, concurrent processes, and data modules. Each component is described in terms of purpose, operation, and dependencies.
- Chapter 4 – Dependency Description
Details the relationships and interactions among the identified modules, processes, and data components, supported by diagrams where appropriate.
- Chapter 5 – Interface Description
Specifies the interfaces exposed by the system's modules and processes, including inputs, outputs, and operational behavior.
- Chapter 6 – Detailed Design
Elaborates on the internal structure of each module and data component, including classes, methods, algorithms, and design patterns applied.
- Appendices
Contain supporting diagrams (use case, class, and sequence diagrams), document evolution notes, and conclusions regarding the development activity.

This structure ensures an organized and comprehensive representation of the SkillHub system, facilitating both implementation and evaluation.

2. References

List of references used in this paper, possible legislation that governs the application domain of the project/document.

[1] IEEE STD-1016-1998, *IEEE Recommended Practice for Software Design Descriptions*.

This standard provides the structural guidelines and recommended practices followed throughout this document.

[2] IEEE STD-830-1998, *IEEE Recommended Practice for Software Requirements Specifications*.

Used as a reference for structuring and validating the Software Requirements Specification (SRS) of the SkillHub system.

[3] SkillHub – Software Requirements Specification (SRS), internal project document.

Defines the functional and non-functional requirements that this design document implements.

[4] Gamma, E., Helm, R., Johnson, R., Vlissides, J. – *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.

Referenced for the application of design patterns used in SkillHub (Factory Method, Singleton, Builder, Facade).

[5] Fowler, M. – *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, Addison-Wesley.

Used as a reference for UML diagrams included in the appendices (use case, class, sequence diagrams).

[6] Goga Nicolae. *Software Engineering – Course Materials*. Politehnica University of Bucharest, 2025.

[7] Dobre, Iulian & Ioniță, Adrian. *Ingineria Programării: Fundamente și Aplicații*. Editura Politehnica Press, București, 2018. ISBN 978-606-515-945-7.

[8] Fiverr – Online Freelancing Platform. Available at: <https://www.fiverr.com>. Accessed: October 2025.

[9] Upwork – Freelancing Platform. Available at: <https://www.upwork.com>. Accessed: October 2025.

3. Decomposition Description

This article corresponds to chapter 6.2.1., Decomposition description, from [1].

3.1. Modules Description

Modules description according to 5.3.1 -5.3.10, from [1]. The modules can be identified on the use case diagram(s) from the Software Requirements Specification.

The system is decomposed into three main functional modules, corresponding to the primary operations defined in the Functional Requirements (FRs).

3.1.1. Description of Module 1: UserManagementModule

Name	UserManagementModule
Type	Code module
Purpose	To handle all user-centric operations: registration, authentication, profile management, and administration of user accounts (corresponds to FR1, FR2, FR11).
Way of operating	Receives requests for registration, login, profile updates (including skills and contact info), and password reset. It validates credentials, interfaces with the UserDataModule for storage, and returns session tokens or profile data. It also exposes specific endpoints for Administrators (FR11) to view/edit/deactivate accounts.
Subordination	<i>None</i> (It is a core module used by the Presentation Layer).
Dependencies	UserDataModule (for data persistence).
Resources	PHP/Python (Back-end framework), bcrypt for password hashing (NFR4).

3.1.2. Description of Module 2: JobSkillManagementModule

Name	JobSkillManagementModule
Type	Code module
Purpose	To manage the creation, modification, search, and submission of proposals for jobs and services (corresponds to FR3, FR4, FR5, FR6).
Way of operating	Clients interact with this module to create/edit/delete job postings (FR3, FR4). Providers interact to search/browse jobs (FR5) using filters (category, budget, keywords) and submit proposals (FR6). It relies on the JobDataModule for persistence and indexing/querying for efficient search (NFR1, NFR5).
Subordination	<i>None</i> (It is a core module used by the Presentation Layer).

Dependencies	JobDataModule (for data persistence).
Resources	PHP/Python (Back-end framework), Search Indexing Service (e.g., Elasticsearch for NFR1/NFR5 performance).

3.1.2. Description of Module 3: CommunicationModerationModule

Name	CommunicationModerationModule
Type	Code module
Purpose	To handle direct messaging between users (Clients/Providers) and to provide tools for Administrators to moderate content (corresponds to FR7 , FR8 , FR10).
Way of operating	Manages the secure, real-time exchange of messages between authenticated users (FR7, FR8). It validates communication permissions based on user roles and project status. Administrators (FR10) access this module's functions to review reported content, proposals, and messages to enforce platform rules. It interfaces with the MessagingDataModule and triggers the NotificationProcess (3.2.1).
Subordination	<i>None</i> (It is a core module used by the Presentation Layer).
Dependencies	MessagingDataModule (for persistence), NotificationProcess (for new message alerts).
Resources	PHP/Python (Back-end framework), WebSockets (or similar tech for real-time messaging).

3.2. Description of Concurrent Processes

Description of concurrent processes according to 5.3.1 – 5.3.10, from [1]. The processes can be identified on the sequence diagram(s) from the Software Requirements Specification.

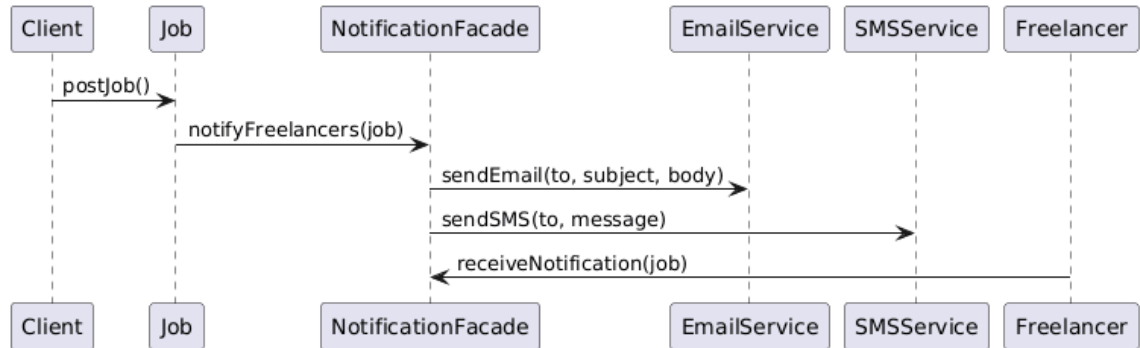
These processes operate independently or are asynchronously triggered by the main functional modules, ensuring timely execution of background and maintenance tasks as required by the system's Functional and Non-Functional Requirements.

These processes run in the background, triggered by system events or scheduled intervals, ensuring non-functional requirements are met.

3.2.1. Description of Process 1: NotificationProcess

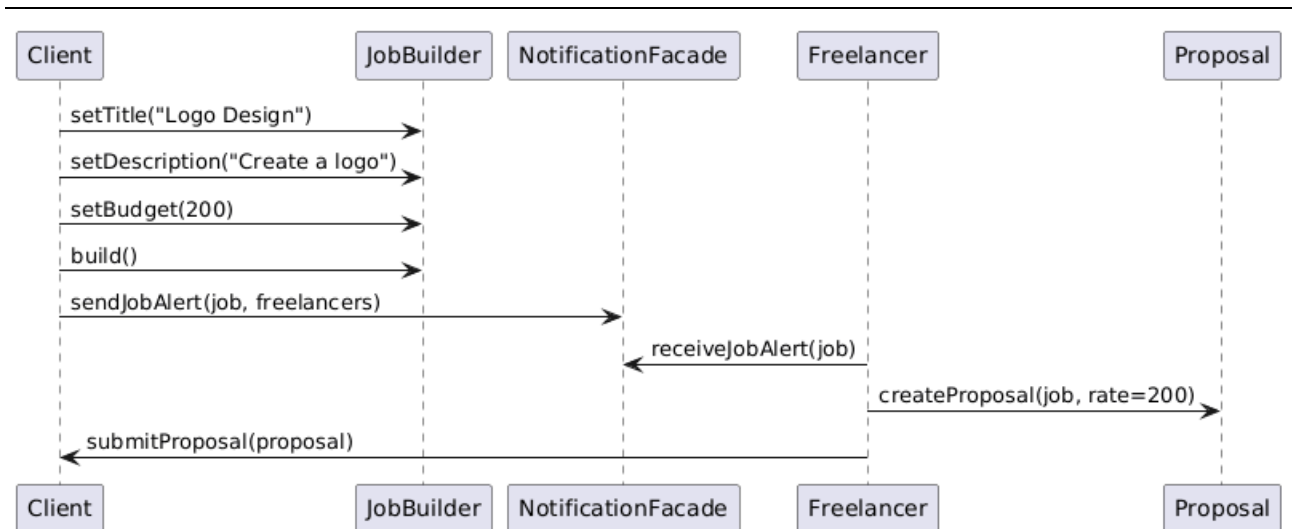
Name	NotificationProcess
Type	Process
Purpose	To asynchronously generate and deliver internal and external

	notifications to users (sms or email) for important events such as new messages, new proposals, or job updates.
Way of operating	The process continuously monitors if a Job is posted, then it accesses the EmailService, sending SMS/Email notification, allowing Freelancers to receive notifications.
Subordination	None
Dependencies	UserDataModule
Resources	Message Queue (e.g., RabbitMQ or Redis) for asynchronous processing, External Email Service/API.



3.2.2. Description of Process 2: Job Process

Name	Job Creation Process
Type	Process
Purpose	To ensure all new and updated job postings are quickly indexed in the specialized Search Indexing Engine, which is necessary to support the high-performance search and filtering required by Providers (FR5, NFR1, NFR5).
Way of operating	This process is triggered when creating a job, which will be notified to all Freelancers so that can be aware of it. Freelancers can create proposals for specific jobs.
Subordination	None
Dependencies	JobDataModule (to read job data), JobSkillManagementModule (as the triggering component).
Resources	Dedicated Search Indexing Engine (e.g., Elasticsearch or similar), Message Queue for receiving index requests.



3.3. Description of Data Modules

Description of data modules according to 5.3.1 – 5.3.10, from [1]. The data modules can be identified on the use case diagram(s) from the Software Requirements Specification.

These modules define the logical structure and operation of the primary data stores required by the functional modules.

3.3.1. Description of Data Module 1: UserDataModule

Name	UserDataModule
Type	Data module
Purpose	To persistently store and manage all user-related information, including credentials, profiles, skills, and admin records (supporting FR1, FR2, FR11, NFR4, NFR10).
Way of operating on data from this module	Provides secure CRUD (Create, Read, Update, Delete) operations. Write operations for sensitive data (like passwords) are handled <i>after</i> the calling module (e.g., UserManagementModule) applies hashing. Read operations support authentication checks and profile viewing. Deletion operations must comply with GDPR (NFR10).
Subordination	<i>None</i>
Dependencies	UserManagementModule, NotificationProcess.
Resources	Relational data base (e.g., MySQL or PostgreSQL, SQL compatible), Secure Storage.

3.3.2. Description of Data Module 2: JobDataModule

Name	JobDataModule
Type	Data module

Purpose	To persistently store and manage all job postings, service listings, proposals, and associated metadata (supporting FR3, FR4, FR5, FR6).
Way of operating on data from this module	Provides CRUD operations for job postings (by Clients) and proposal submissions (by Providers). Supports complex read operations for searching and filtering based on keywords, category, and budget (required by FR5). Efficient indexing is necessary for high-performance searches (NFR1, NFR5).
Subordination	<i>None</i>
Dependencies	JobSkillManagementModule.
Resources	Relational data base (SQL compatible), Search Indexing Engine.

3.3.3. Description of Data Module 3: MessagingDataModule

Name	MessagingDataModule
Type	Data module
Purpose	To persistently store communication records, ensuring message history between Clients and Providers is maintained and accessible for moderation (supporting FR7, FR8, FR10).
Way of operating on data from this module	Primarily supports write operations for new messages and read operations to display conversation history. It must ensure that only authenticated and authorized users (the two parties involved, plus Administrators for moderation) can access the data. Data transfer and storage must be secure (NFR4).
Subordination	<i>None</i>
Dependencies	CommunicationModerationModule.
Resources	Relational data base or NoSQL store (e.g., MongoDB for high message volume).

4. Dependency Description

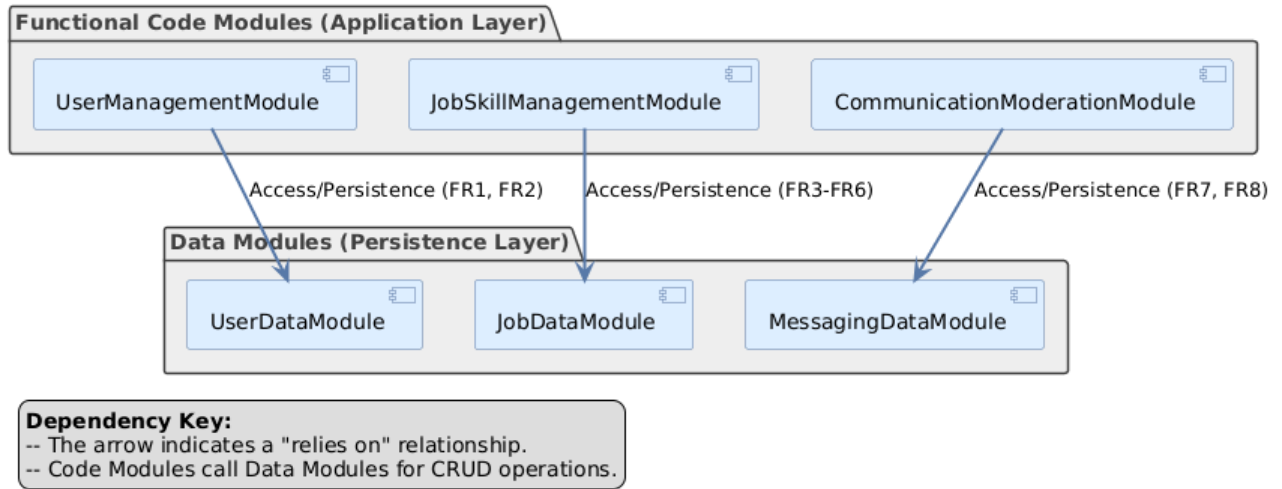
This chapter corresponds to chapter 6.2.2, *Dependency description*, from [1].

This chapter details the dependencies among the decomposed components (Modules, Concurrent Processes, and Data Modules) of the SkillHub system, as identified in Chapter 3.

4.1. Dependencies among modules

The dependencies identified in chapter 3.1 are detailed. Detailed diagrams are recommended.

SkillHub Functional Module Dependencies (Chapter 4.1)



The functional Code Modules rely on the Data Modules for persistence. These dependencies represent direct calls from the application logic to the data access layer.

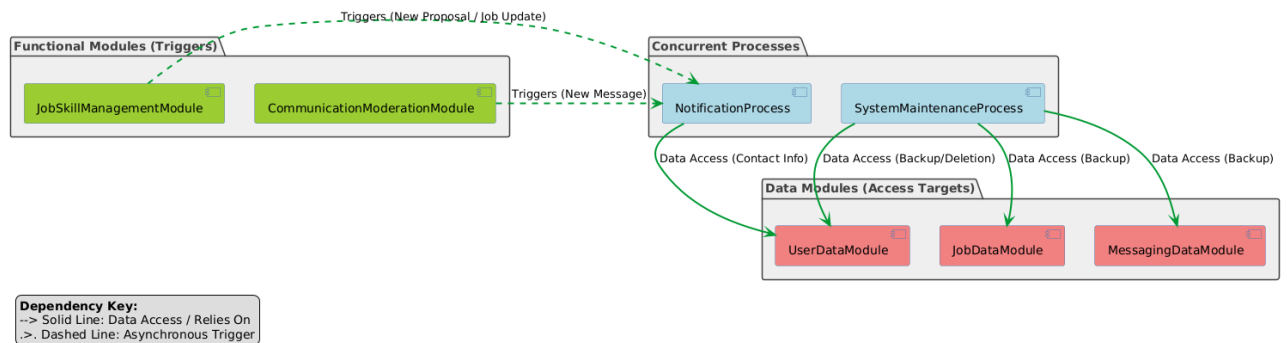
Dependent Module	Dependency Type	Target Module(s)	Description
UserManagementModule	Access/Persistence	UserDataModule	Requires access to store, retrieve, update, and delete user accounts and profile details (FR1, FR2, FR11).
JobSkillManagementModule	Access/Persistence	JobDataModule	Requires access to manage job postings, proposals, and perform complex searches on job/skill data (FR3-FR6).

Dependent Module	Dependency Type	Target Module(s)	Description
CommunicationModerationModule	Access/Persistence	MessagingDataModule	Requires access to store and retrieve all user-to-user messages for conversation history and moderation (FR7, FR8, FR10).

4.2. Dependencies among processes

The dependencies identified in chapter 3.2 are detailed. Detailed diagrams are recommended.

SkillHub Concurrent Process Dependencies (Chapter 4.2)



Concurrent Processes often interact with functional modules and data modules to perform background tasks, or they can trigger each other.

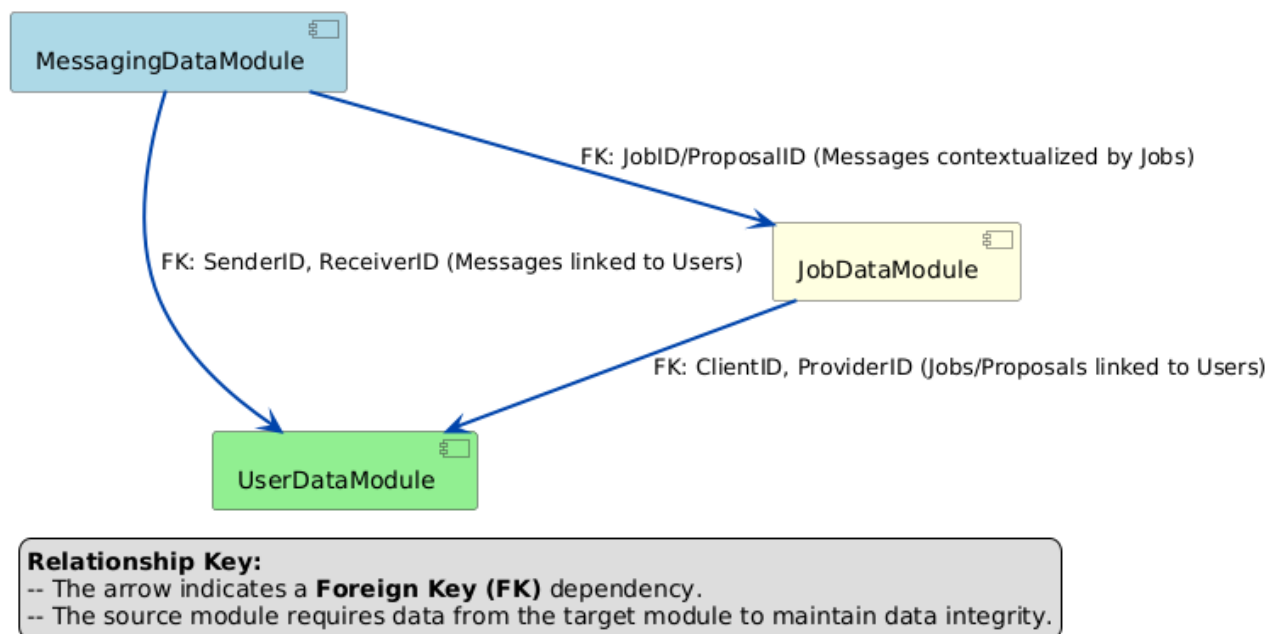
Dependent Process	Dependency Type	Target Component(s)	Description
NotificationProcess	Triggered By	CommunicationModerationModule and JobSkillManagementModule	The Notification Process is <i>triggered</i> when either the communication module records a new message (FR9) or the job module records a new proposal/job update.
NotificationProcess	Data Access	UserDataModule	Needs user contact information (e.g.,

Dependent Process	Dependency Type	Target Component(s)	Description
			email address) to send external notifications (FR9).
SystemMaintenanceProcess	Data Access	UserDataModule, JobDataModule, MessagingDataModule	Needs read access to all three data modules to perform daily backups (NFR6) and needs write access to the UserDataModule to handle GDPR-compliant permanent data deletion (NFR10).

4.3. Dependencies among data modules

The dependencies identified in chapter 3.3 are detailed. Detailed diagrams are recommended.

SkillHub Data Module Dependencies (Chapter 4.3)



The next crucial step in your SDD is to detail the dependencies between the modules, processes, and data stores you defined in Section 3. This ensures the communication and integration points are clearly documented.

Here is the completed **Chapter 4: Dependency Description** for SkillHub.

Dependent Data Module	Relationship Type	Target Data Module(s)	Description
JobDataModule	Foreign Key (FK)	UserDataModule	Jobs must be posted by a registered Client (FK: ClientID). Proposals must be submitted by a registered Provider (FK: ProviderID).
MessagingDataModule	Foreign Key (FK)	UserDataModule and JobDataModule	Messages must be linked to the sending and receiving User (FK: SenderID, ReceiverID). Messages are usually contextualized by a Job or Proposal (FK: JobID/ProposalID).

5. Interface Description

This chapter corresponds to chapter 6.2.3, *Interface description*, from [1].

This chapter details the external and internal interfaces for the Modules and Concurrent Processes defined in Chapter 3. These interfaces define the specific function signatures, inputs, and outputs necessary for component interaction.

5.1. Module Interfaces

The module interfaces described in chapter 3.1 are detailed. Detailed diagrams are recommended.

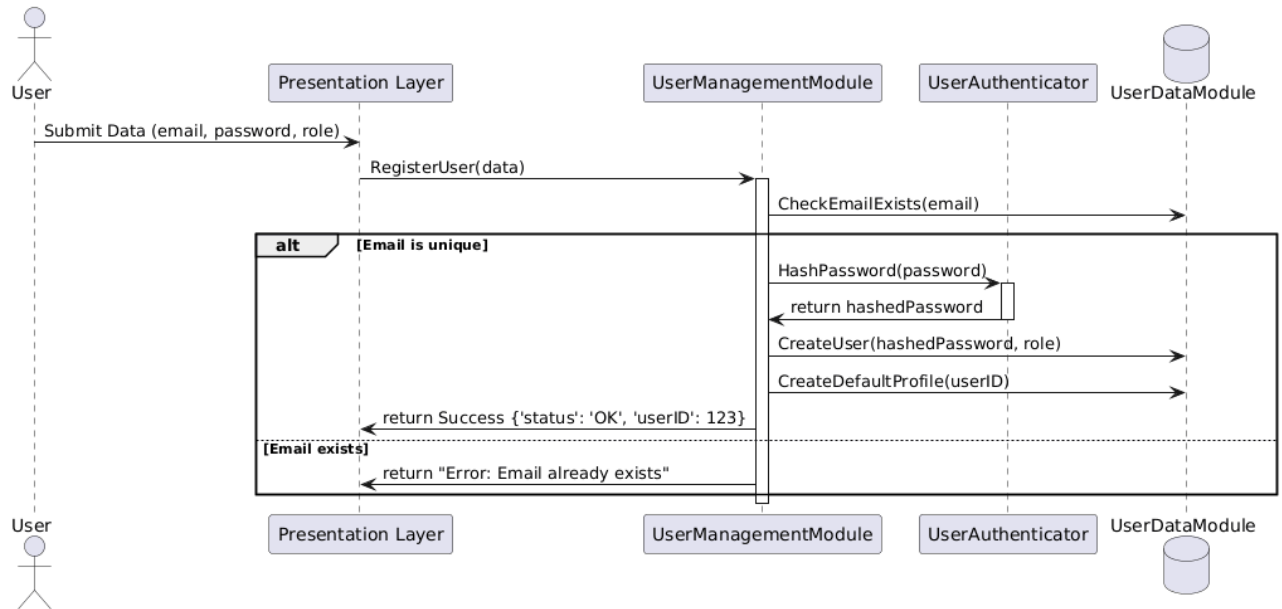
The functional modules expose an Application Programming Interface (API) to the Presentation Layer (front-end) and other internal components.

5.1.1. Module 1 Interface: **UserManagementModule**

Name	UserManagementModule
Type	Code module
Purpose	To handle all user-centric operations: registration, authentication, profile management, and administration of user accounts.
Way of operating	Receives requests for registration, login, profile updates, and password reset. Validates credentials, interfaces with the UserDataModule for storage, and returns session tokens or profile data.
Interface 1	result RegisterUser(email, password, role)
Input	email: string – Unique user email. password: string – User access password (will be hashed internally). role: enum ('Client' or 'Provider') – Initial user role.
Output	result: object/string – User ID and a success/failure status (e.g., {'status': 'OK', 'userID': 123}). Failure includes a reason (e.g., 'Email already exists').
Description	Handles the creation of a new user account. It verifies that the email is unique, hashes the password (NFR4), and persists the data via the UserDataModule .
Interface 2	result Login(email, password)
Input	email: string – User email. password: string – User access password.
Output	result: object/string – Session token/ID and user details upon successful authentication, or a 'FAILED' status.
Description	Authenticates the user by comparing the provided password against the stored hash. If successful, it returns a token for subsequent secure access (FR1, NFR4).
Interface 3	result UpdateProfile(userID, profileData)

Input	userID: integer – The ID of the user modifying the profile. profileData: object – Contains fields like name, description, skills (array of strings), and contact information.
Output	result: boolean – true if update was successful, false otherwise.
Description	Allows a user to edit their personal profile, skills, and contact details (FR2).

5.1.1 Interface 1: RegisterUser (FR1, NFR4)

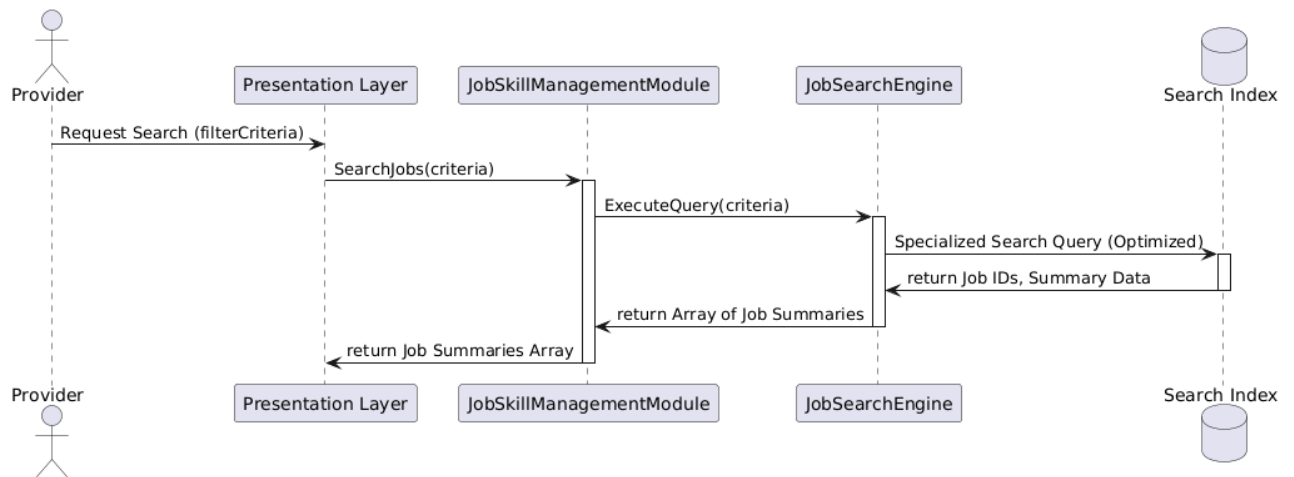


5.1.2. Module 2 Interface: JobSkillManagementModule

Name	JobSkillManagementModule
Type	Code module
Purpose	To manage the creation, modification, search, and submission of proposals for jobs and services.
Way of operating	Clients create/manage jobs (FR3, FR4). Providers search jobs (FR5) and submit proposals (FR6). Interfaces with the JobDataModule .
Interface 1	result PostJob(clientID, jobDetails)
Input	clientID: integer – ID of the client posting the job. jobDetails: object – Includes title, description, budget: decimal, deadline: date (FR3).
Output	result: object/string – The newly created Job ID and a status (e.g., {'status': 'OK', 'jobID': 456}).
Description	Creates and publishes a new job posting by persisting the details in the JobDataModule .

Interface 2	array SearchJobs(filterCriteria)
Input	filterCriteria: object – Contains search parameters like category: string, minBudget: decimal, keywords: string, etc. (FR5).
Output	array: An array of job summary objects that match the criteria.
Description	Executes a high-performance search query across the job listings. Must return results within the performance bounds (NFR1).
Interface 3	result SubmitProposal(providerID, jobID, proposalDetails)
Input	providerID: integer – ID of the provider submitting the proposal. jobID: integer – ID of the target job. proposalDetails: object – Includes proposedPrice: decimal, deliveryTime: integer (days), description: string (FR6).
Output	result: boolean – true if the proposal was successfully recorded, false otherwise.
Description	Records a provider's bid for a specific job, persisting it in the JobDataModule and triggering a notification to the client (Dependency to NotificationProcess).

5.1.2 Interface 2: SearchJobs (FR5, NFR1)

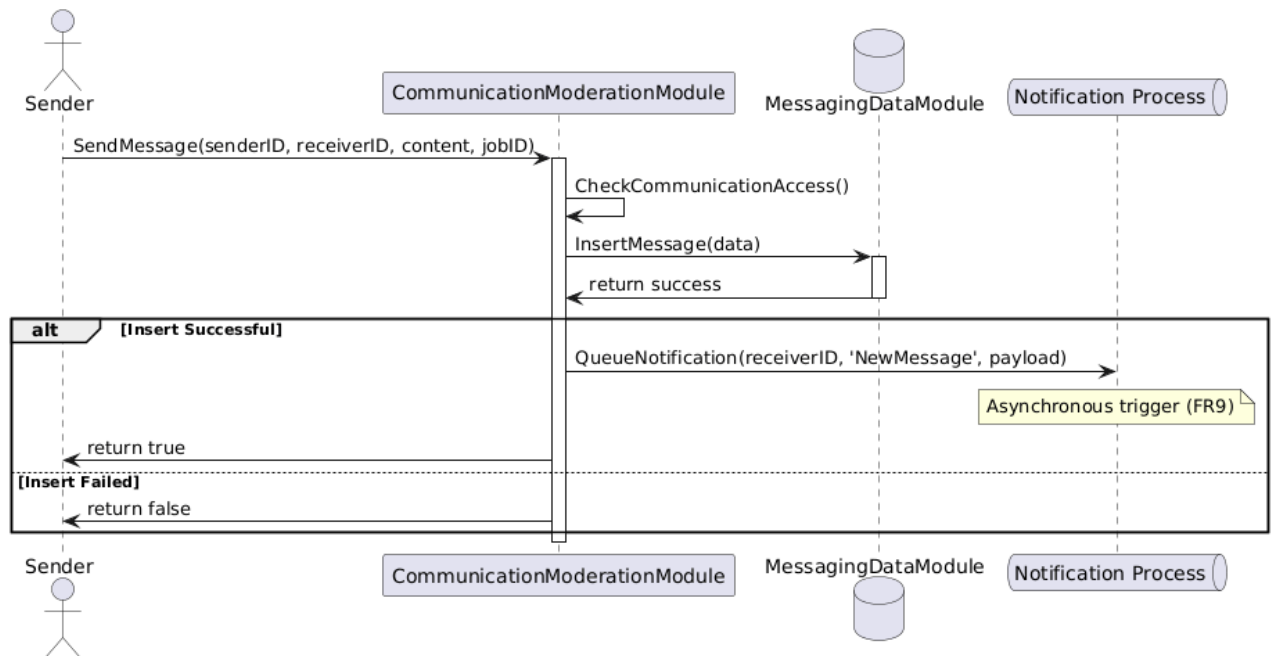


5.1.3. Module 3 Interface: CommunicationModerationModule

Name	CommunicationModerationModule
Type	Code module
Purpose	To handle direct messaging between users and to provide tools for Administrators to moderate content.
Way of operating	Manages secure message exchange (FR7, FR8). Administrators access tools to review and enforce rules (FR10).
Interface 1	result SendMessage(senderID, receiverID, content, jobID)

Input	senderID: integer, receiverID: integer – IDs of the communicating users. content: string – The message body. jobID: integer – Contextual job ID (optional).
Output	result: boolean – true if the message was stored and notification triggered, false otherwise.
Description	Stores the message securely (NFR4) and uses WebSockets/APIs for potential real-time delivery. Triggers the NotificationProcess (FR9).
Interface 2	array GetConversation(userID1, userID2, jobID)
Input	userID1, userID2: integers – IDs of the two users. jobID: integer – Contextual job ID.
Output	array: Array of message objects (sorted chronologically).
Description	Retrieves the secure message history between two users related to a specific job (FR7, FR8).
Interface 3	result ModerateContent(adminID, contentID, action)
Input	adminID: integer – ID of the acting administrator. contentID: integer – ID of the message/proposal being reviewed. action: enum ('Remove', 'WarnUser', 'DeactivateUser').
Output	result: boolean – true if the action was successfully logged and executed.
Description	Exposes administrative tools to enforce platform rules (FR10) by reviewing and acting upon inappropriate content.

5.1.3 Interface 1: SendMessage (FR7, FR9)

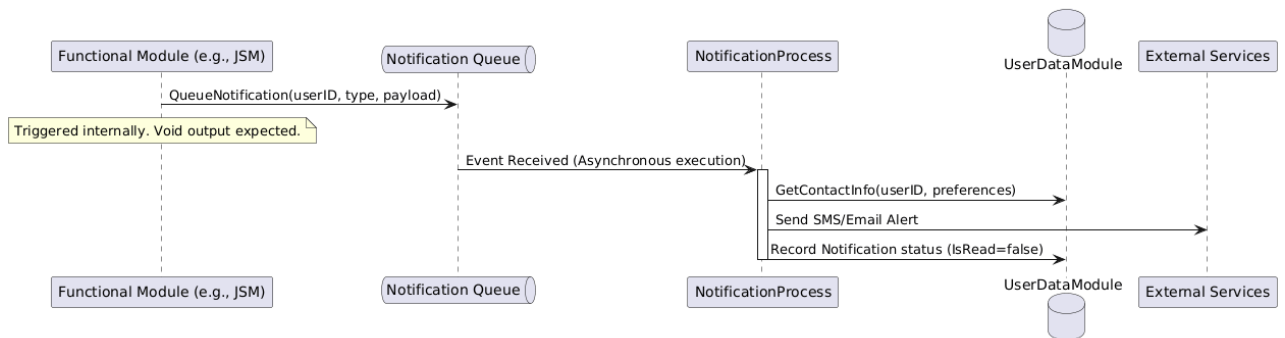


5.2. Processes Interfaces

5.2.1. Process 1 Interface: NotificationProcess

Name	NotificationProcess
Type	Process
Purpose	To asynchronously generate and deliver internal and external notifications (SMS or email) for important events.
Interface 1	void QueueNotification(userID, type, payload)
Input	userID: integer – The target user ID. type: enum ('NewMessage', 'NewProposal', 'JobUpdate', 'NewJobPosted'). payload: object – Contextual data (e.g., job title, sender name).
Output	None (Void function).
Description	This is the generic internal trigger used by functional modules (e.g., Communication or Job Skill Management) to queue an alert. The process handles asynchronous delivery via email/SMS (as per the module description) after fetching user contact data from the UserDataModule.

5.2.1 Process 1 Interface: QueueNotification (FR9)

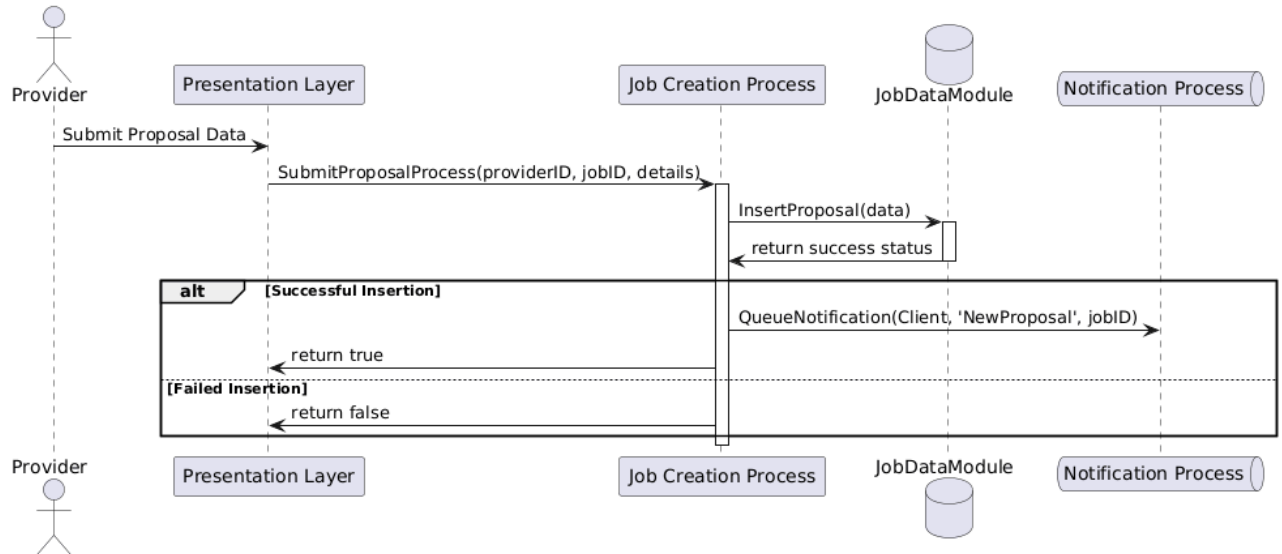


5.2.2. Process 2 Interface: Job Creation Process

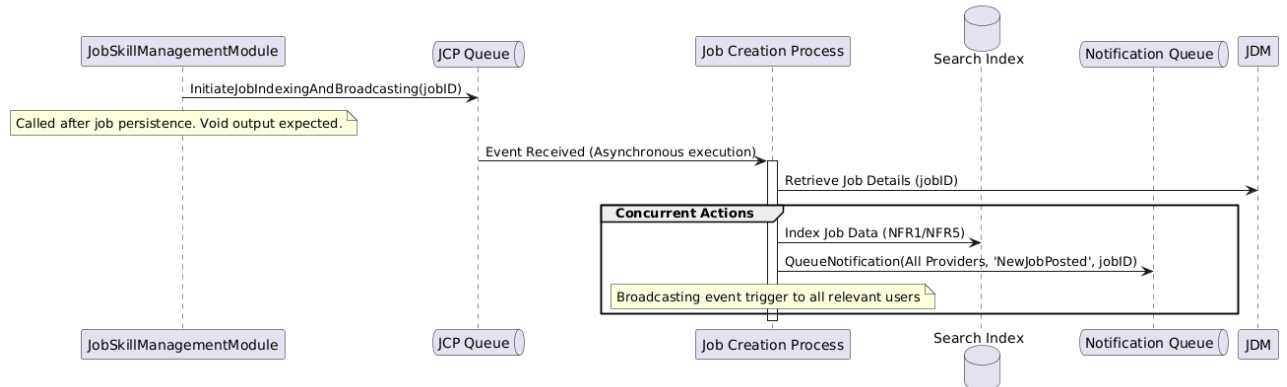
Name	Job Creation Process
Type	Process
Purpose	To ensure new jobs are quickly indexed for search and to manage the broad notification of new jobs to all relevant Freelancers (Providers).
Interface 1	void InitiateJobIndexingAndBroadcasting(jobID)
Input	jobID: integer – The ID of the newly created job.
Output	None (Void function).
Description	This interface is called by the JobSkillManagementModule immediately after a job is successfully posted. It queues the job for two concurrent actions: 1) Indexing in the Search Engine (for NFR1/NFR5 performance). 2) Broadcasting the 'New Job Posted' event, which triggers the NotificationProcess for applicable Freelancers.
Interface 2	result SubmitProposalProcess(providerID, jobID, proposalDetails)
Input	providerID: integer, jobID: integer, proposalDetails: object (price, time, description).

Output	result: boolean – true if the proposal was successfully recorded, false otherwise.
Description	Although named a 'Process,' this interface is essentially the API for a Provider to submit a proposal, which is an asynchronous trigger to the job process to record the proposal and initiate subsequent notifications to the client. This mirrors the behavior described in the 'Way of operating' section (Freelancers can create proposals).

5.2.2 Process 2 Interface: SubmitProposalProcess (FR6)



5.2.2 Process 2 Interface: InitiateJobIndexing (NFR1, NFR5)



6. Detailed Design

This chapter corresponds to chapter 6.2.3, *Interface description*, from [1].

6.1. Modules detailed design

The modules described in chapter 3.1 are detailed. Detailed diagrams are recommended.

6.1.1. Module 1: UserManagementModule

Name	UserManagementModule
Type	Code module
Purpose	To handle all user-centric operations: registration, authentication, profile management, and administration of user accounts (FR1, FR2, FR11).
Way of operating	Receives API requests, validates data, delegates persistence to the Data Access Layer (DAL), and manages user state (session/token).
Classes	UserController, UserAuthenticator, UserProfileManager

6.1.1.1. Module 1, Class 1: UserController

Name	UserController
Purpose	Acts as the entry point for all external (API) user management requests. It handles input validation and delegates core logic to UserAuthenticator and UserProfileManager.
Members	authenticator: UserAuthenticator, profileManager: UserProfileManager, logger: SystemLogger
Methods	RegisterUser, Login, UpdateProfile, AdminManageAccount (Mapping to external interfaces from 5.1.1)

6.1.1.1.1. Module 1, Class 1, Method 1: RegisterUser

Name	RegisterUser
Purpose	To create a new user account with basic credentials and role.
Prototype	result RegisterUser(string email, string password, string role)
Input	email: string, password: string, role: string ('Client' or 'Provider')
Output	result: object/string – User ID and status.
Caller	Presentation Layer / API Gateway.

Calls	UserAuthenticator.HashPassword, UserDataModule.CreateUser, UserProfileManager.CreateDefaultProfile.
Algorithm	1. Validate Input: Check for valid email format and password strength. 2. Check Uniqueness: Call UserDataModule.CheckEmailExists(email). If exists, return 'Email already exists'. 3. Hash Password: Call UserAuthenticator.HashPassword(password) (using bcrypt as per NFR4). 4. Create User: Call UserDataModule.CreateUser(email, hashedPassword, role). 5. Create Profile: Call UserProfileManager.CreateDefaultProfile(newUserID). 6. Return: Return new User ID and 'OK' status.

6.1.1.2. Module 1, Class 2: UserAuthenticator

Name	UserAuthenticator
Purpose	Handles all security-sensitive operations, specifically password hashing and user login verification.
Members	saltService: SecurityService
Methods	HashPassword, VerifyPassword, GenerateSessionToken

6.1.1.2.1. Module 1, Class 2, Method 1: VerifyPassword

Name	VerifyPassword
Purpose	To securely verify a user's login password against the stored hash.
Prototype	boolean VerifyPassword(string email, string inputPassword)
Input	email: string, inputPassword: string (password provided by the user).
Output	boolean: true if passwords match, false otherwise.
Caller	UserController.Login.
Calls	UserDataModule.GetStoredHash(email).
Algorithm	1. Fetch Hash: Call UserDataModule.GetStoredHash(email) to retrieve the stored bcrypt hash. 2. Check Hash: If no hash is found, return false. 3. Compare: Use the bcrypt comparison library function to securely compare inputPassword against the stored hash (NFR4). 4. Return: Return the result of the comparison.

6.1.1.3. Module 1, Class 3: UserProfileManager

Name	UserProfileManager
-------------	--------------------

Purpose	Manages the non-authentication-related user data, such as profiles, skills, and contact information.
Members	<i>None</i>
Methods	CreateDefaultProfile, UpdateProfile, RetrieveProfile, SanitizeProfileData

6.1.1.3.1. Module 1, Class 3, Method 1: UpdateProfile

Name	UpdateProfile
Purpose	To update a user's professional profile details, including name, description, and skills.
Prototype	boolean UpdateProfile(integer userID, object profileData)
Input	userID: integer, profileData: object (data fields for update).
Output	boolean: true if update successful, false otherwise.
Caller	UserController.UpdateProfile.
Calls	UserProfileManager.SanitizeProfileData, UserDataModule.UpdateProfileRecord.
Algorithm	1. Authorization Check: Verify the userID is the owner of the profile being updated. 2. Sanitize Data: Call SanitizeProfileData(profileData) to clean input (e.g., prevent XSS attacks). 3. Validate: Check if all required fields are present and valid (e.g., skills are strings). 4. Persist: Call UserDataModule.UpdateProfileRecord(userID, sanitizedData) (FR2). 5. Return: Return success status from the persistence operation.

6.1.2. Module 2: JobSkillManagementModule

Name	JobSkillManagementModule
Type	Code module
Purpose	To manage the creation, modification, search, and submission of proposals for jobs and services (FR3, FR4, FR5, FR6).
Way of operating	Receives requests from Clients to post jobs and from Providers to search jobs and submit proposals. It validates project data and uses the JobDataModule for persistence.
Classes	JobController, JobPoster, JobSearchEngine

6.1.2.1. Module 2, Class 1: JobController

Name	JobController
-------------	---------------

Purpose	Acts as the entry point for all external API requests related to job postings and proposals. It delegates complex read/write logic to the JobPoster and JobSearchEngine.
Members	jobPoster: JobPoster, searchEngine: JobSearchEngine, logger: SystemLogger
Methods	PostJobAPI, ManagePostedJobsAPI, BrowseJobsAPI, SubmitProposalAPI

6.1.2.2. Module 2, Class 2: JobPoster

Name	JobPoster
Purpose	Encapsulates the logic for creating, editing, deleting jobs (Client side) and handling proposal submissions (Provider side) (Write operations) (FR3, FR4, FR6).
Members	dataAccess: JobDataModule
Methods	CreateJob, UpdateJob, DeleteJob, RecordProposal

6.1.2.2.1. Module 2, Class 2, Method 1: RecordProposal

Name	RecordProposal
Purpose	To validate and persist a Provider's proposal for a specific job, and trigger the client notification.
Prototype	result RecordProposal(integer providerID, integer jobID, object proposalDetails)
Input	providerID: integer, jobID: integer, proposalDetails: object (includes proposed price, delivery time, description).
Output	result: boolean – true if successful, false otherwise.
Caller	JobController.SubmitProposalAPI.
Calls	JobDataModule.GetJobStatus, JobDataModule.InsertProposal, NotificationProcess.TriggerNotification.
Algorithm	1. Validate Job Status: Call JobDataModule.GetJobStatus(jobID). If the job is closed or awarded, return false. 2. Validate Proposal: Check for non-empty price, positive delivery time, and sanitize description. 3. Persist Proposal: Call JobDataModule.InsertProposal(providerID, jobID, details) (FR6). 4. Trigger Notification: If persistence is successful, call NotificationProcess.TriggerNotification(clientID, 'NewProposal', payload) to alert the client (FR9). 5. Return: Return persistence success status.

6.1.2.3. Module 2, Class 3: JobSearchEngine

Name	JobSearchEngine
Purpose	Manages the efficient searching, filtering, and retrieval of job postings, optimizing for performance and scalability (Read operations) (FR5, NFR1, NFR5).
Members	searchIndexClient: SearchIndexingService, dataAccess: JobDataModule
Methods	SearchJobs, GetJobDetails

6.1.2.3.1. Module 2, Class 3, Method 1: SearchJobs

Name	SearchJobs
Purpose	To quickly find and return available job postings based on complex filtering criteria, meeting response time and scalability requirements.
Prototype	array SearchJobs(object filterCriteria)
Input	filterCriteria: object – Includes category, minBudget, keywords, location, page, pageSize.
Output	array: A paginated array of brief job summary objects.
Caller	JobController.BrowseJobsAPI.

Calls	SearchIndexingService.ExecuteQuery.
Algorithm	1. Index Query Construction: Convert filterCriteria into a specialized query language (e.g., Lucene query) suitable for the search index client. 2. Execute Search: Call SearchIndexingService.ExecuteQuery(query). This step must be highly optimized to return results ≤ 3 seconds (NFR1) under heavy load (NFR5). The search index (not the primary database) is used for this step. 3. Retrieve Details (Optional): The search index returns primary keys (Job IDs). If summary data is missing, perform a batch lookup against the JobDataModule using these IDs. 4. Format Results: Apply pagination (page, pageSize) and structure the output array for the client. 5. Return: Return the array of paginated job summaries.

6.1.3. Module 3: CommunicationModerationModule

Name	CommunicationModerationModule
Type	Code module
Purpose	To handle secure direct messaging between users and provide tools for Administrators to moderate content (FR7, FR8, FR10).
Way of operating	Receives requests to send messages, stores them securely via the MessagingDataModule , retrieves conversation history, and exposes specific interfaces for Administrator oversight. It also triggers the NotificationProcess on new messages.
Classes	MessageController, MessageService, ModerationManager

6.1.3.1. Module 3, Class 1: MessageController

Name	MessageController
Purpose	Serves as the primary API endpoint for all user messaging functions (sending, receiving, and viewing history). It delegates core logic to MessageService.
Members	messageService: MessageService, moderationManager: ModerationManager
Methods	SendMessageAPI, GetConversationAPI

6.1.3.2. Module 3, Class 2: MessageService

Name	MessageService
Purpose	Manages the secure storage, retrieval, and delivery of messages. Ensures communication is tied to relevant users and jobs (FR7, FR8).
Members	dataAccess: MessagingDataModule, notificationProcess: NotificationProcess
Methods	StoreMessage, RetrieveHistory, CheckCommunicationAccess

6.1.3.2.1. Module 3, Class 2, Method 1: StoreMessage

Name	StoreMessage
Purpose	To securely validate, timestamp, and persist a new message between two users and trigger a notification.
Prototype	result StoreMessage(integer senderID, integer receiverID, string content, integer jobID)
Input	senderID: integer, receiverID: integer, content: string, jobID: integer (optional context).
Output	result: boolean – true if storage and notification trigger were successful.
Caller	MessageController.SendMessageAPI.
Calls	MessageService.CheckCommunicationAccess, MessagingDataModule.InsertMessage, NotificationProcess.TriggerNotification.
Algorithm	1. Check Access: Call CheckCommunicationAccess(senderID, receiverID, jobID) to ensure both users are authorized to communicate (e.g., related to the same job). If not authorized,

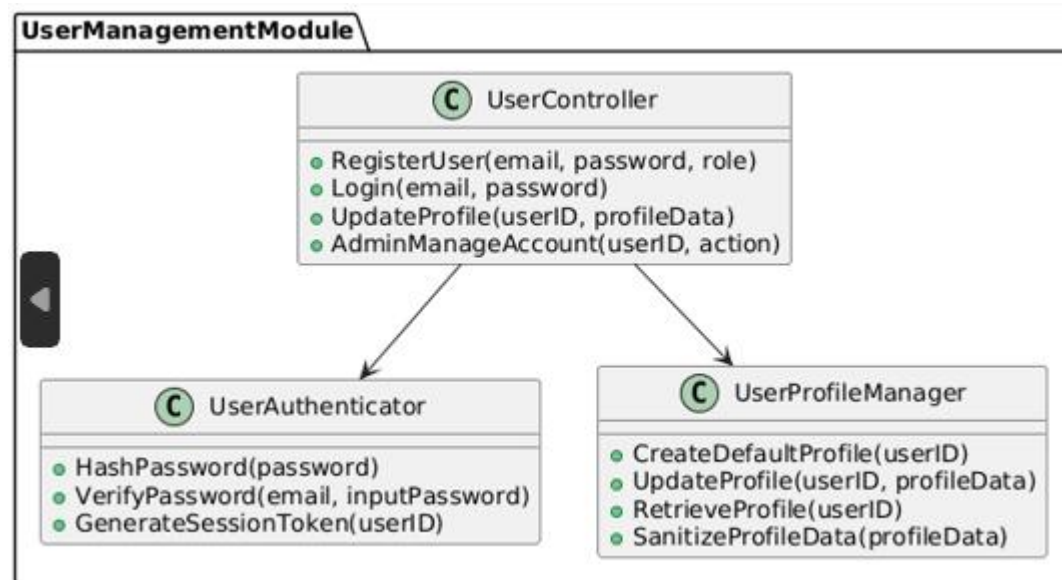
	return false. 2. Sanitize Content: Filter out harmful content (basic XSS/SQL injection prevention). 3. Persist: Call MessagingDataModule.InsertMessage(senderID, receiverID, content, timestamp, jobID) (FR7, FR8). 4. Notify: If persistence is successful, call NotificationProcess.TriggerNotification(receiverID, 'NewMessage', payload) to alert the recipient (FR9). 5. Return: Return persistence success status.
--	--

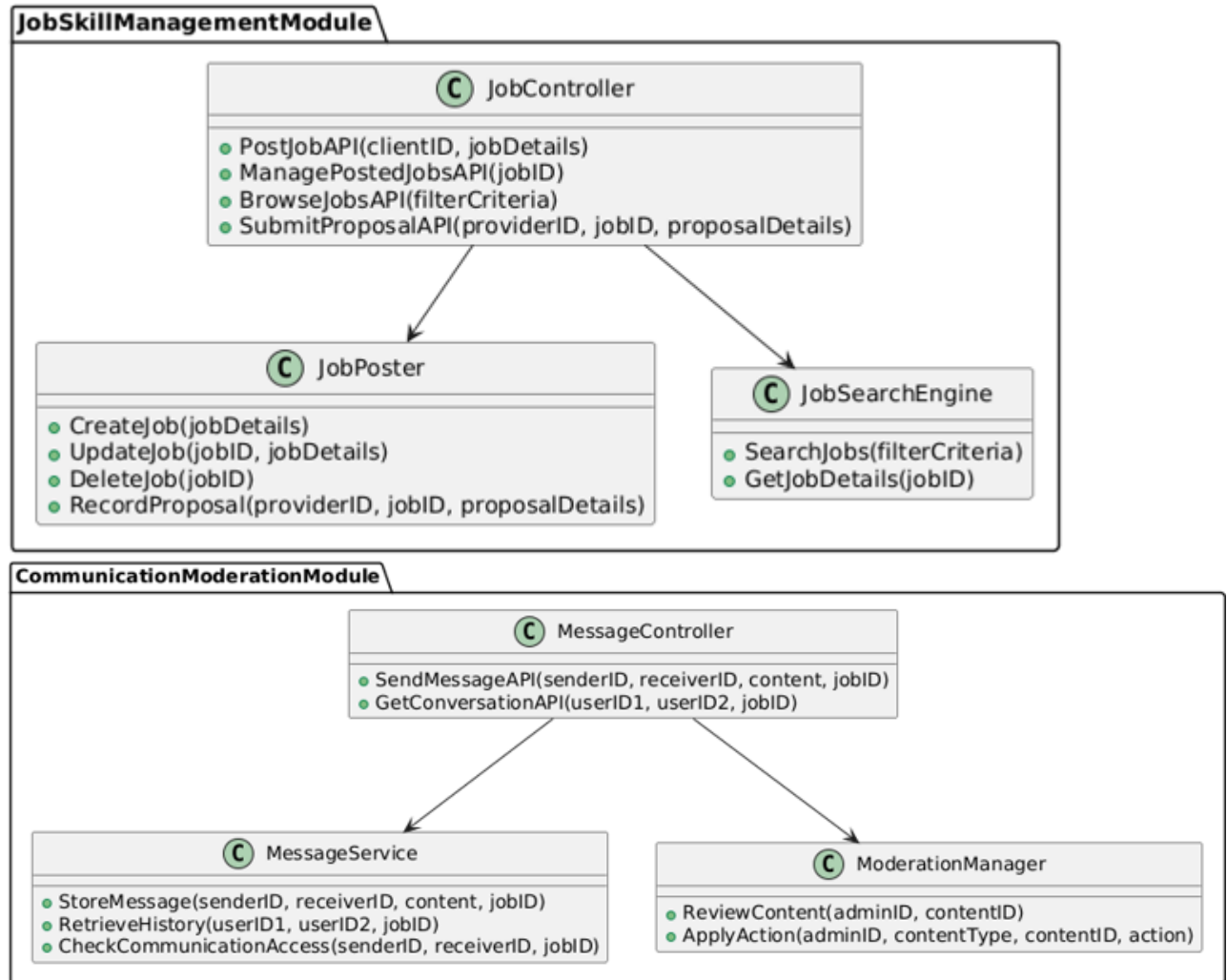
6.1.3.3. Module 3, Class 3: ModerationManager

Name	ModerationManager
Purpose	Provides tools and logic for Administrators to review platform content (messages, proposals, profiles) and enforce platform policies (FR10).
Members	dataAccess: MessagingDataModule, userManagement: UserManagementModule
Methods	ReviewContent, ApplyAction

6.1.3.3.1. Module 3, Class 3, Method 1: ApplyAction

Name	ApplyAction
Purpose	To execute an administrative action (remove, warn, deactivate) against a piece of content or a user account (FR10).
Prototype	result ApplyAction(integer adminID, string contentType, integer contentID, string action)
Input	adminID: integer (must be authenticated Admin), contentType: string ('Message', 'Proposal', 'Profile'), contentID: integer, action: string ('Remove', 'WarnUser', 'DeactivateUser').
Output	result: boolean – true if the action was executed and logged.
Caller	MessageController (Admin interface endpoint).
Calls	MessagingDataModule.FlagContent (or DeleteMessage), JobDataModule.FlagContent, UserManagementModule.AdminManageAccount (for 'DeactivateUser').
Algorithm	1. Admin Verification: Verify adminID has administrative privileges. If not, return false. 2. Action Delegation: Use a switch statement based on action: - If ' Remove ': Call the appropriate Data Module to logically delete/flag the content. - If ' DeactivateUser ': Call UserManagementModule.AdminManageAccount(targetUserID, 'Deactivate') (FR11). - If ' WarnUser ': Log the warning and trigger a specific notification to the user. 3. Log Action: Record the adminID, action, and contentID for audit purposes. 4. Return: Return true.





6.2. Data Modules Detailed Design

The data modules described in chapter 3.3. are detailed. Detailed diagrams are recommended.

This section details the internal structure and organization of the Data Modules, describing the key entities (tables), their attributes (columns), and the relationships between them.

6.2.1. Data Module 1: UserDataModule

This module manages all user accounts and profile information, supporting authentication and user management (FR1, FR2, FR11, NFR4, NFR10).

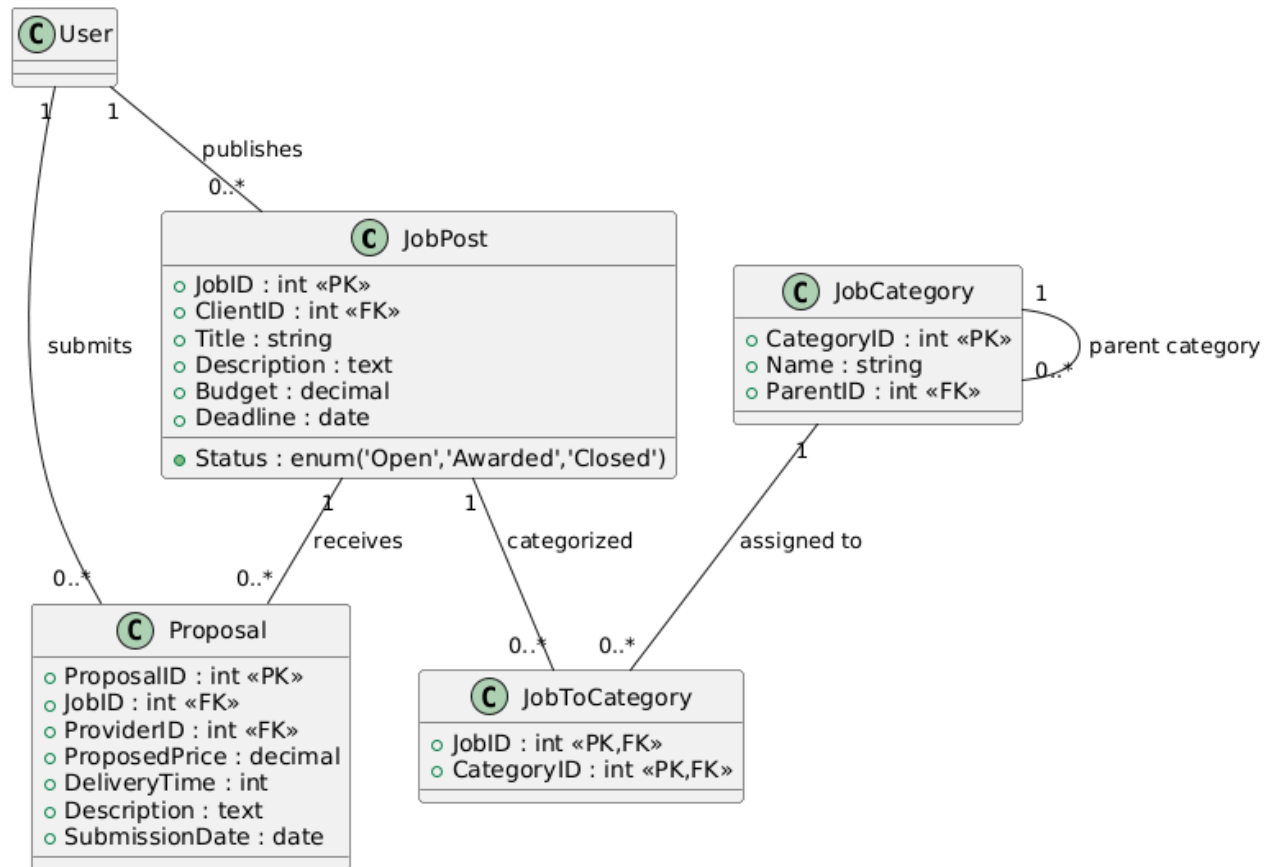
Entity (Table)	Purpose & Key Fields	Attributes (Columns)	Constraints / Notes
User	Stores core login credentials and user type.	UserID (PK), Email (Unique, Not Null), PasswordHash (Not Null), Role ('Client', 'Provider', 'Admin'), IsActive (Boolean)	NFR4: PasswordHash uses bcrypt. IsActive supports Admin deactivation (FR11).
Profile	Stores descriptive information for Client and Provider profiles.	ProfileID (PK), UserID (FK to User), Name (Not Null), Description, ContactInfo, Location, RegistrationDate	FR2: Supports user profile creation and editing.

Entity (Table)	Purpose & Key Fields	Attributes (Columns)	Constraints / Notes
Skill	Stores a list of specific skills or categories offered by Providers.	SkillID (PK), Name (Unique), Category	Used for taxonomy and search filtering (FR5).
UserSkill	Maps Providers to the Skills they offer (Many-to-Many).	UserID (FK, PK part), SkillID (FK, PK part), ExperienceLevel	Supports Provider profile skill listing.
AdminLog	Records all administrative actions for audit purposes.	LogID (PK), AdminID (FK to User), ActionType, TargetUserID, Timestamp	Supports platform moderation and oversight (FR10, FR11).

6.2.2. Data Module 2: JobDataModule

This module manages all job postings, service listings, and the associated proposals, supporting project initiation and searching (FR3, FR4, FR5, FR6, NFR1, NFR5).

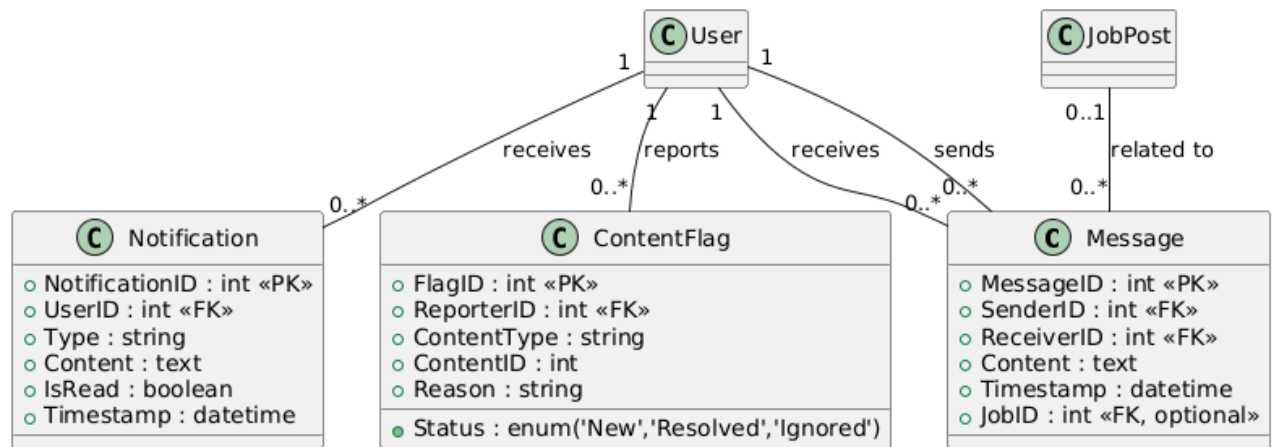
Entity (Table)	Purpose & Key Fields	Attributes (Columns)	Constraints / Notes
JobPost	Stores job requests published by Clients.	JobID (PK), ClientID (FK to User), Title (Not Null), Description (Text), Budget (Decimal), Deadline (Date), Status ('Open', 'Awarded', 'Closed')	FR3: Supports job posting. Required fields are indexed for fast searching (NFR1, NFR5).
Proposal	Stores bids submitted by Providers for open jobs.	ProposalID (PK), JobID (FK to JobPost), ProviderID (FK to User), ProposedPrice (Decimal), DeliveryTime (Days), Description (Text), SubmissionDate	FR6: Supports proposal submission.
JobCategory	Stores the taxonomy used for job organization.	CategoryID (PK), Name (Unique), ParentID (FK to self)	Used for filtering job search results (FR5).
JobToCategory	Maps JobPosts to one or more Categories (Many-to-Many).	JobID (FK, PK part), CategoryID (FK, PK part)	Supports multi-category filtering.



6.2.3. Data Module 3: MessagingDataModule

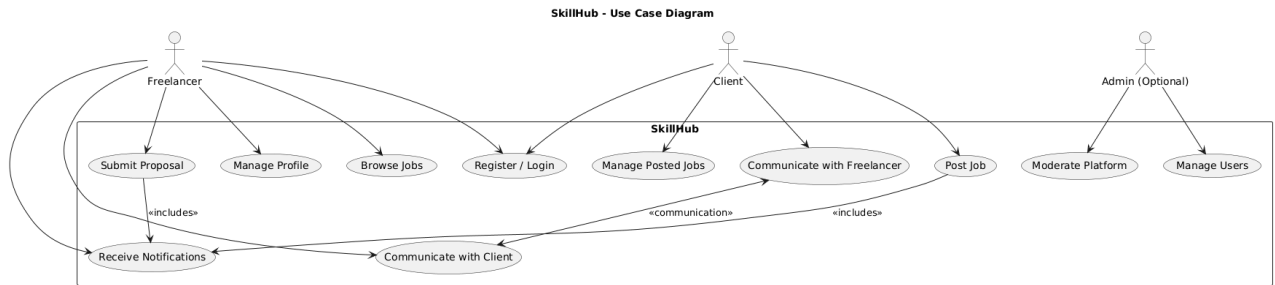
This module manages all communication records, ensuring security and accessibility for moderation (FR7, FR8, FR10, NFR4).

Entity (Table)	Purpose & Key Fields	Attributes (Columns)	Constraints / Notes
Message	Stores the content of direct messages between users.	MessageID (PK), SenderID (FK to User), ReceiverID (FK to User), Content (Text, Secured), Timestamp (Not Null), JobID (FK to JobPost, optional)	FR7, FR8: Supports client/provider communication. NFR4: Encryption may be applied to Content if required by legal standards beyond HTTPS.
Notification	Stores the history and status of system alerts.	NotificationID (PK), UserID (FK to User), Type ('NewMessage', 'NewProposal', etc.), Content, IsRead (Boolean), Timestamp	FR9: Supports the notification process.
ContentFlag	Records instances of inappropriate content flagged by users or system checks.	FlagID (PK), ReporterID (FK to User), ContentType ('Message', 'Proposal', etc.), ContentID, Reason, Status ('New', 'Resolved', 'Ignored')	FR10: Supports the moderation process.



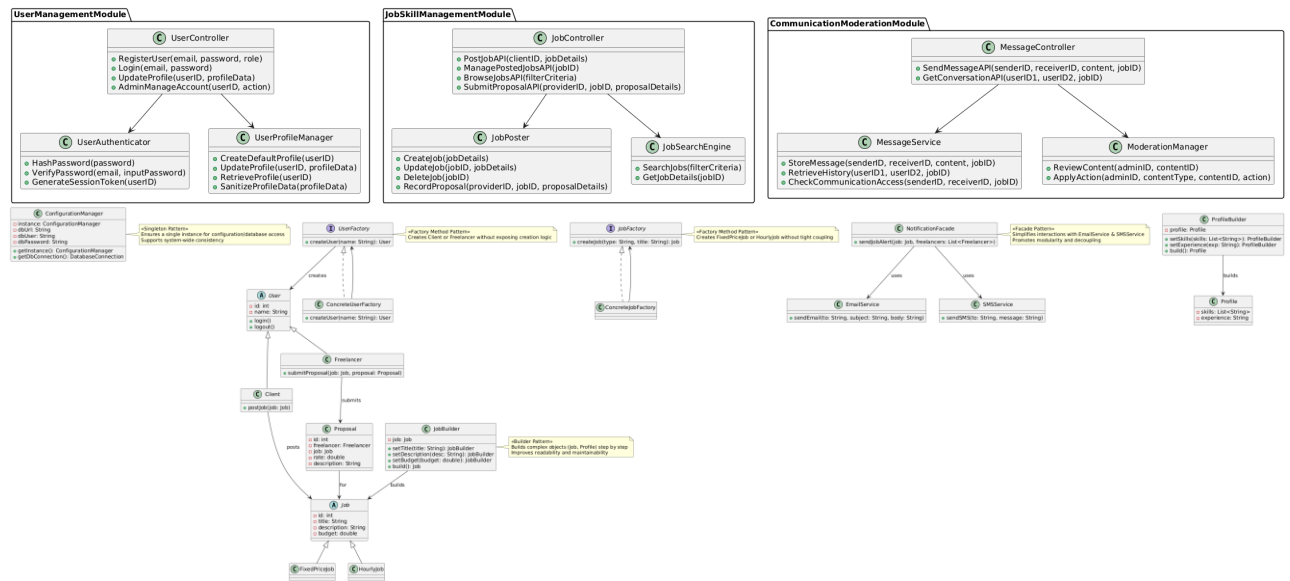
Appendices

A1. Use cases diagrams

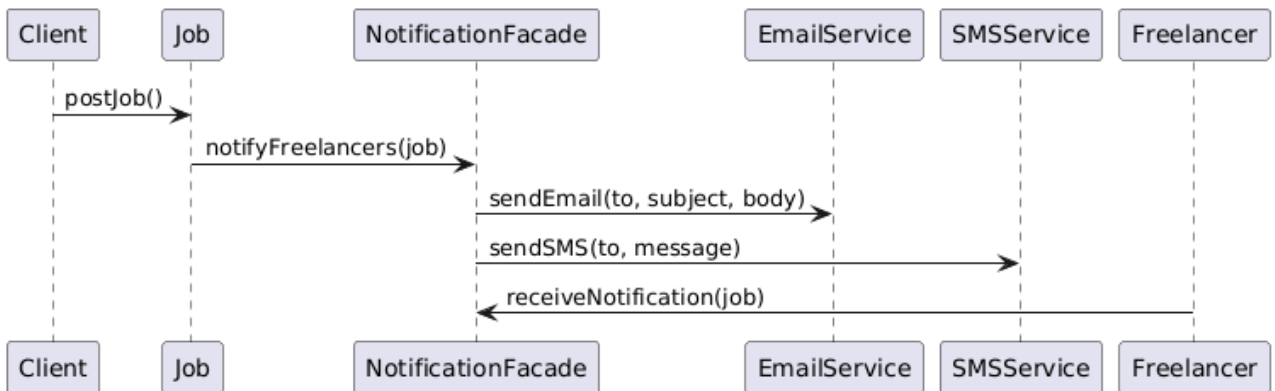


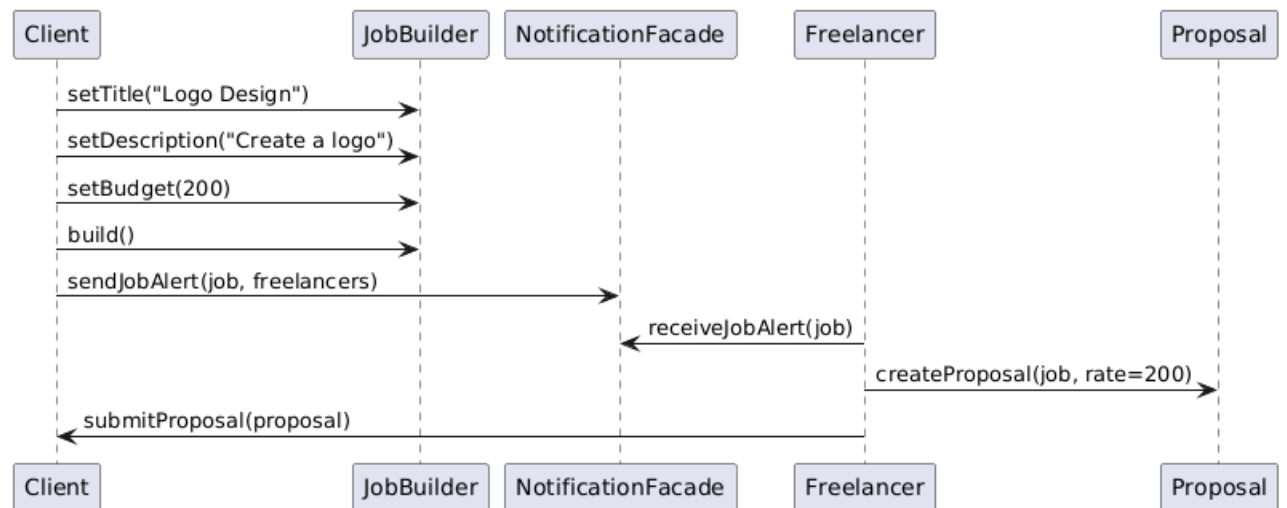
A2. Class diagrams

6.1.1



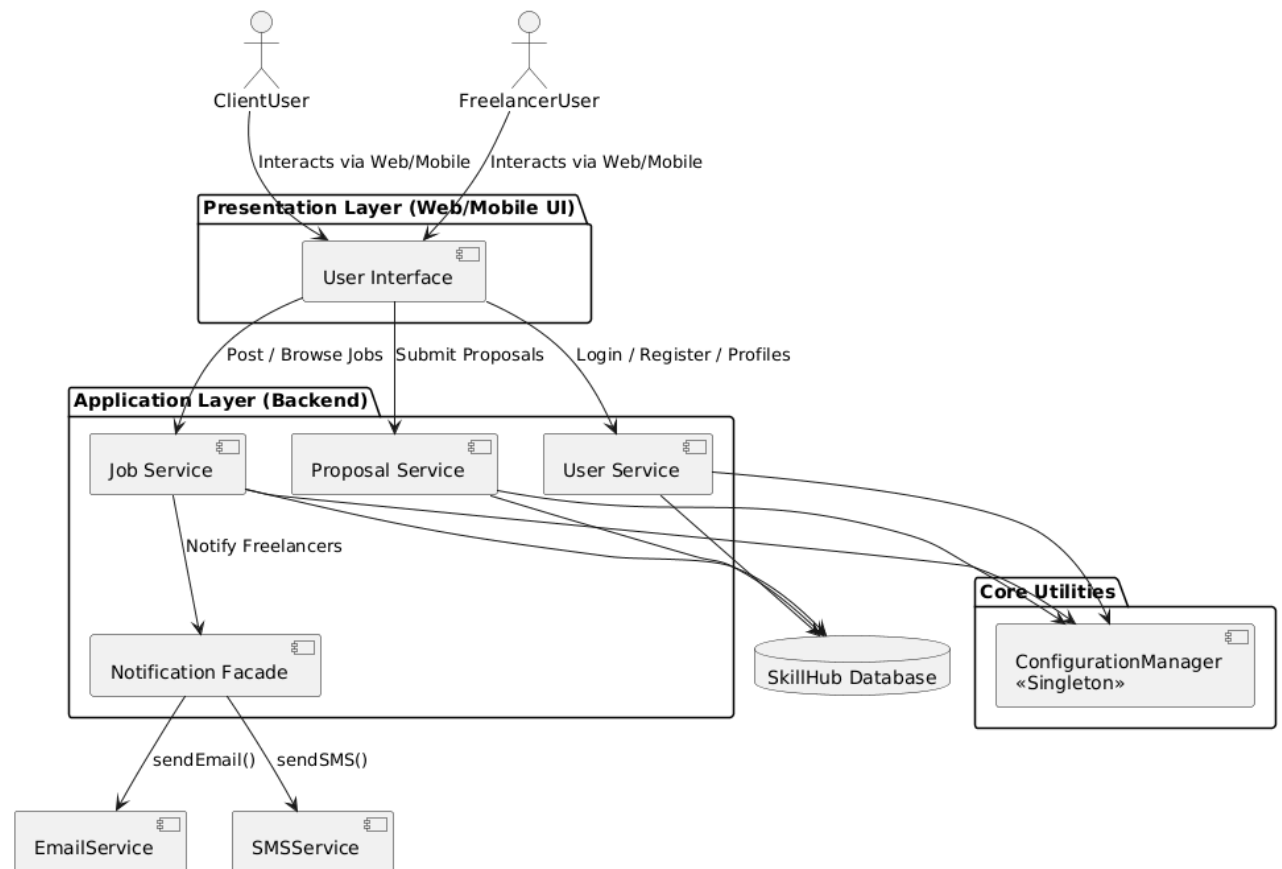
A3. Sequence diagrams





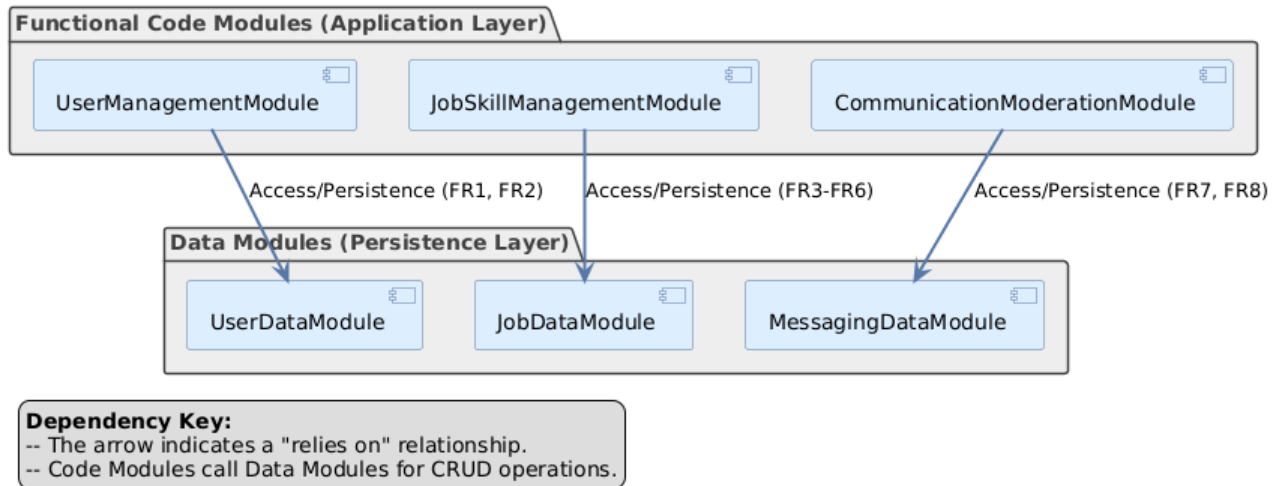
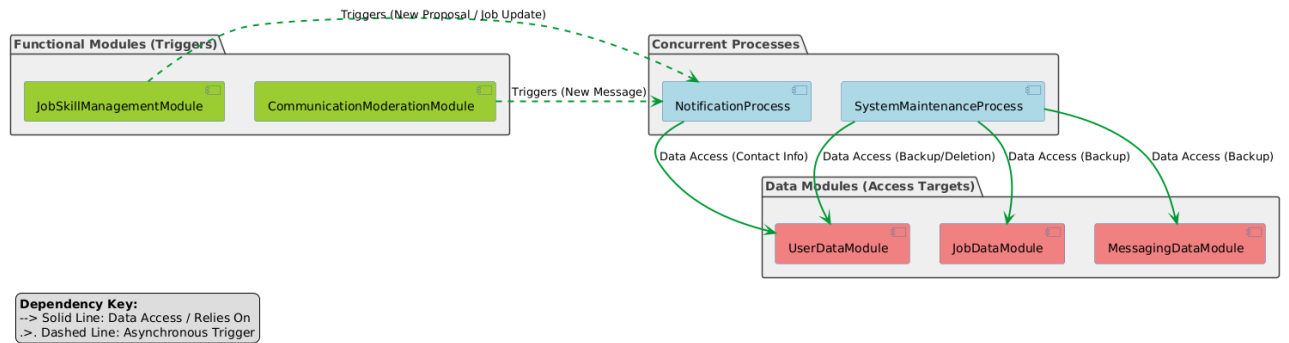
A4. System Diagram

SkillHub System Architecture (Component Diagram)

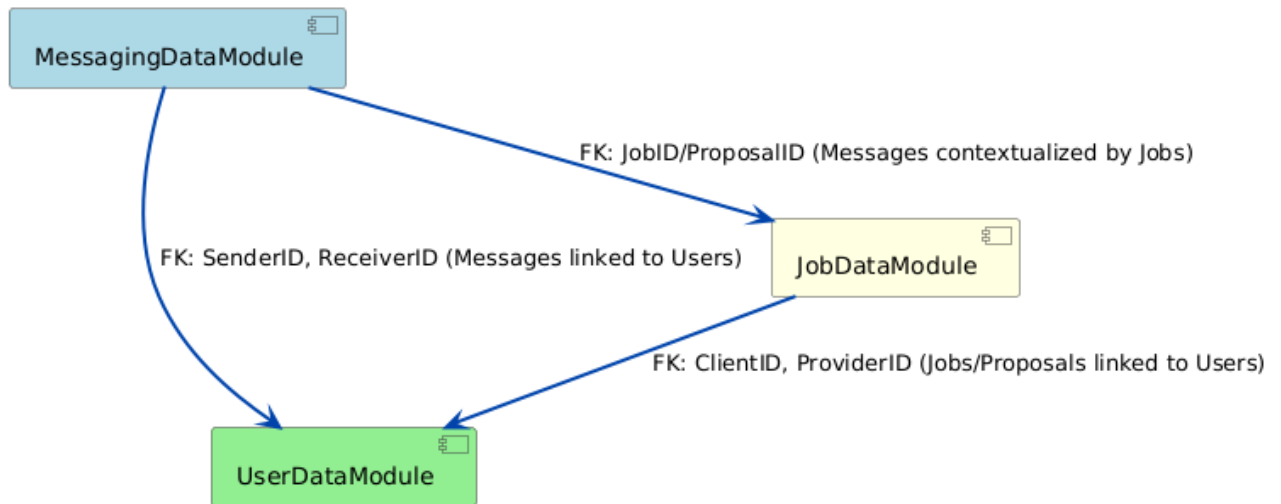


A5. Diagrams

4.1

SkillHub Functional Module Dependencies (Chapter 4.1)**4.2****SkillHub Concurrent Process Dependencies (Chapter 4.2)****4.3**

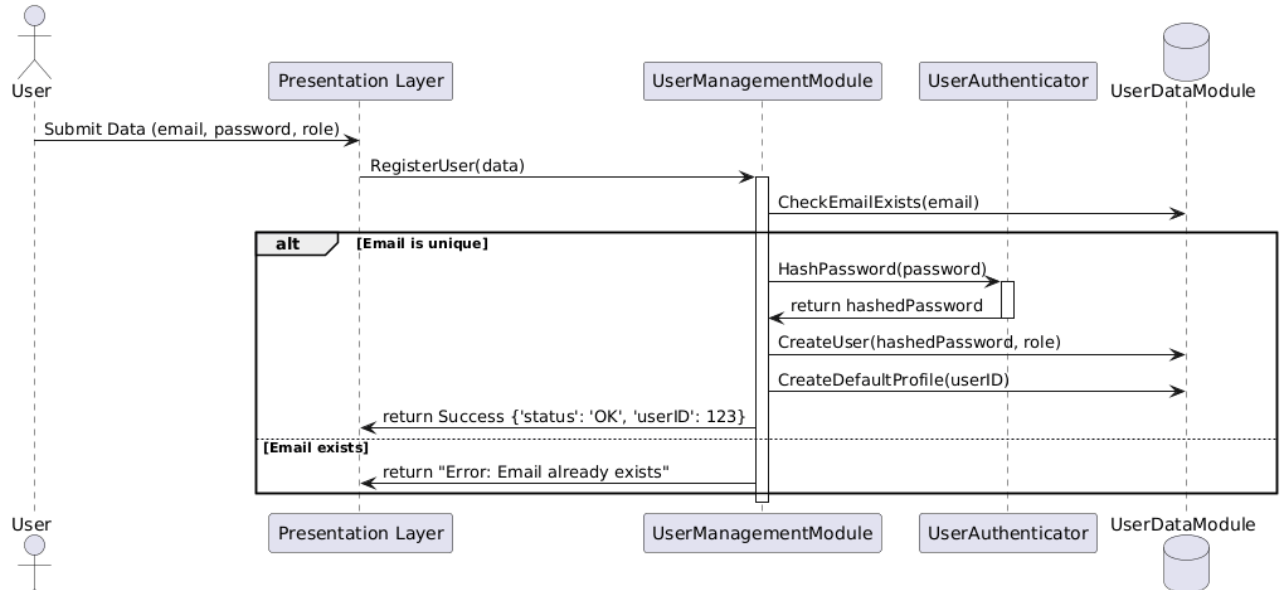
SkillHub Data Module Dependencies (Chapter 4.3)


Relationship Key:

- The arrow indicates a **Foreign Key (FK)** dependency.
- The source module requires data from the target module to maintain data integrity.

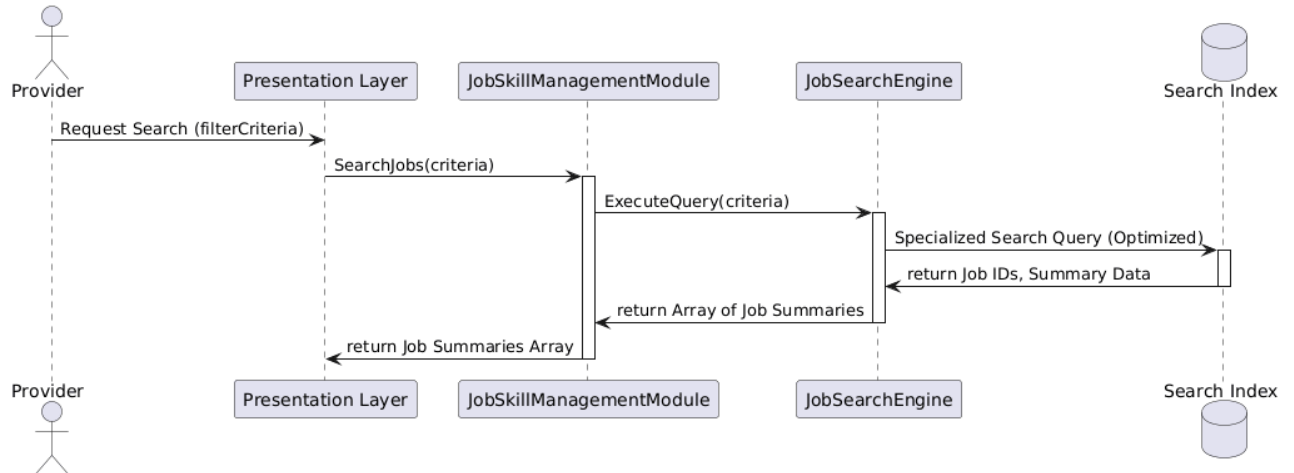
5.1.1

5.1.1 Interface 1: RegisterUser (FR1, NFR4)



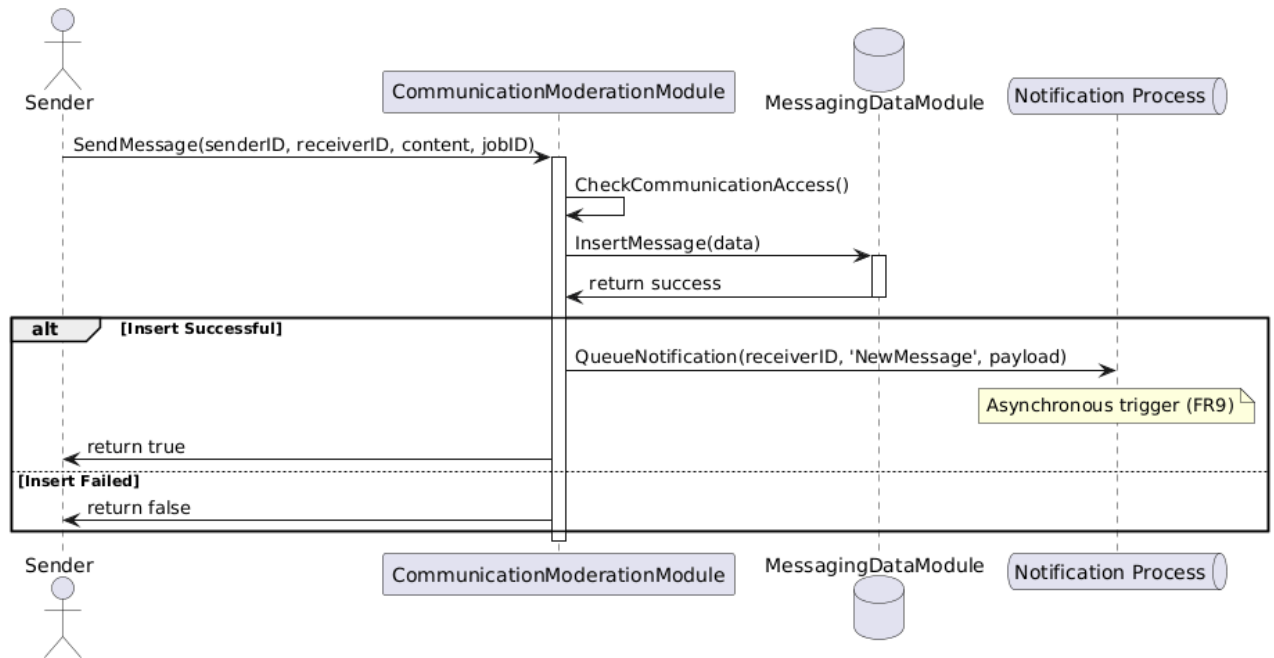
5.1.2

5.1.2 Interface 2: SearchJobs (FR5, NFR1)



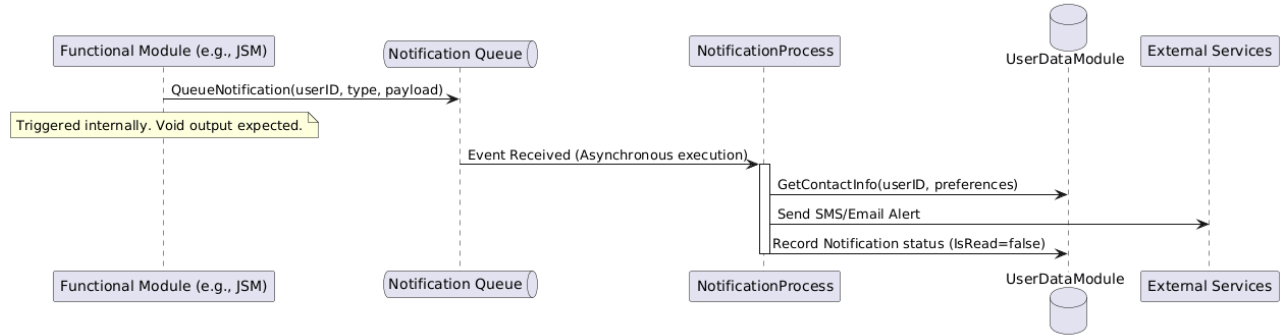
5.1.3

5.1.3 Interface 1: SendMessage (FR7, FR9)



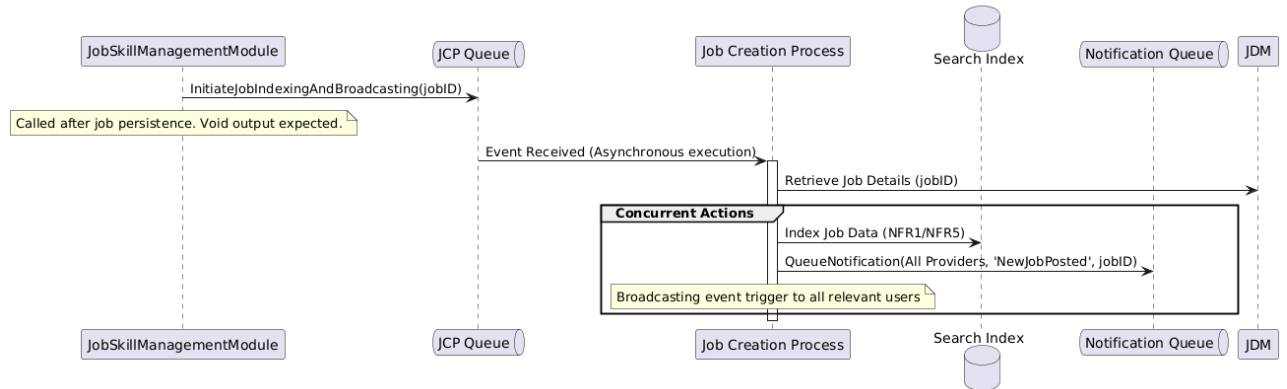
5.2.1

5.2.1 Process 1 Interface: QueueNotification (FR9)

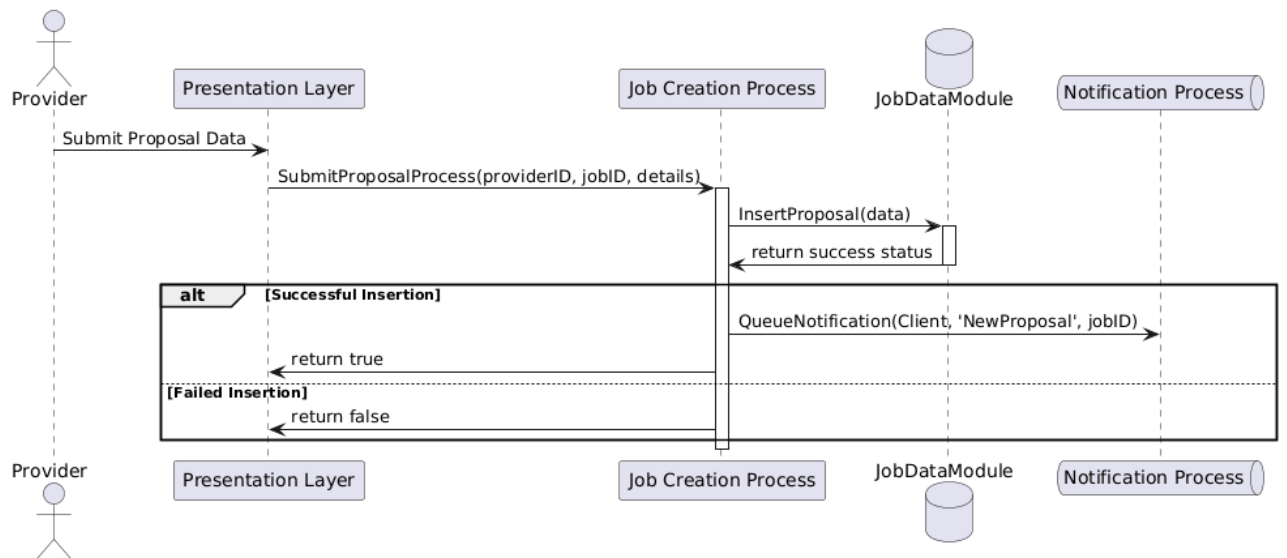


5.2.2(1 and 2)

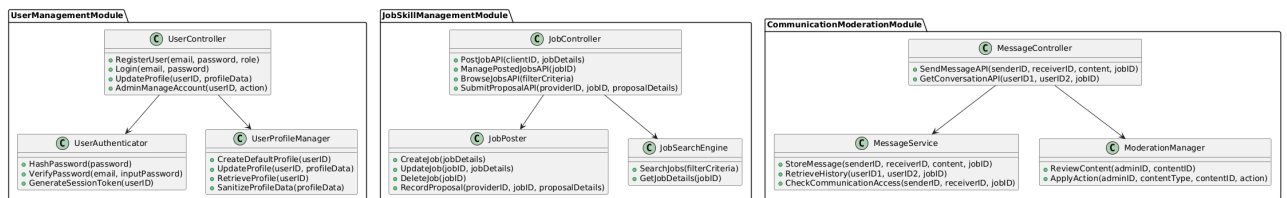
5.2.2 Process 2 Interface: InitiateJobIndexing (NFR1, NFR5)



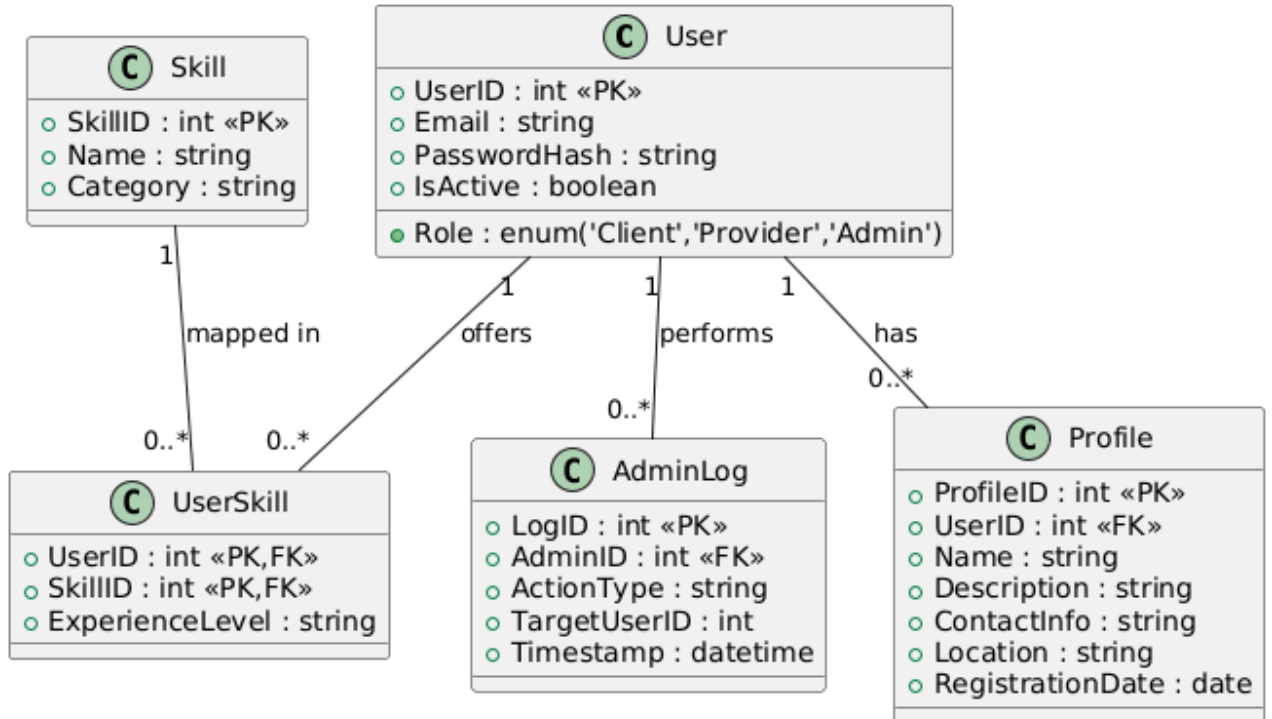
5.2.2 Process 2 Interface: SubmitProposalProcess (FR6)



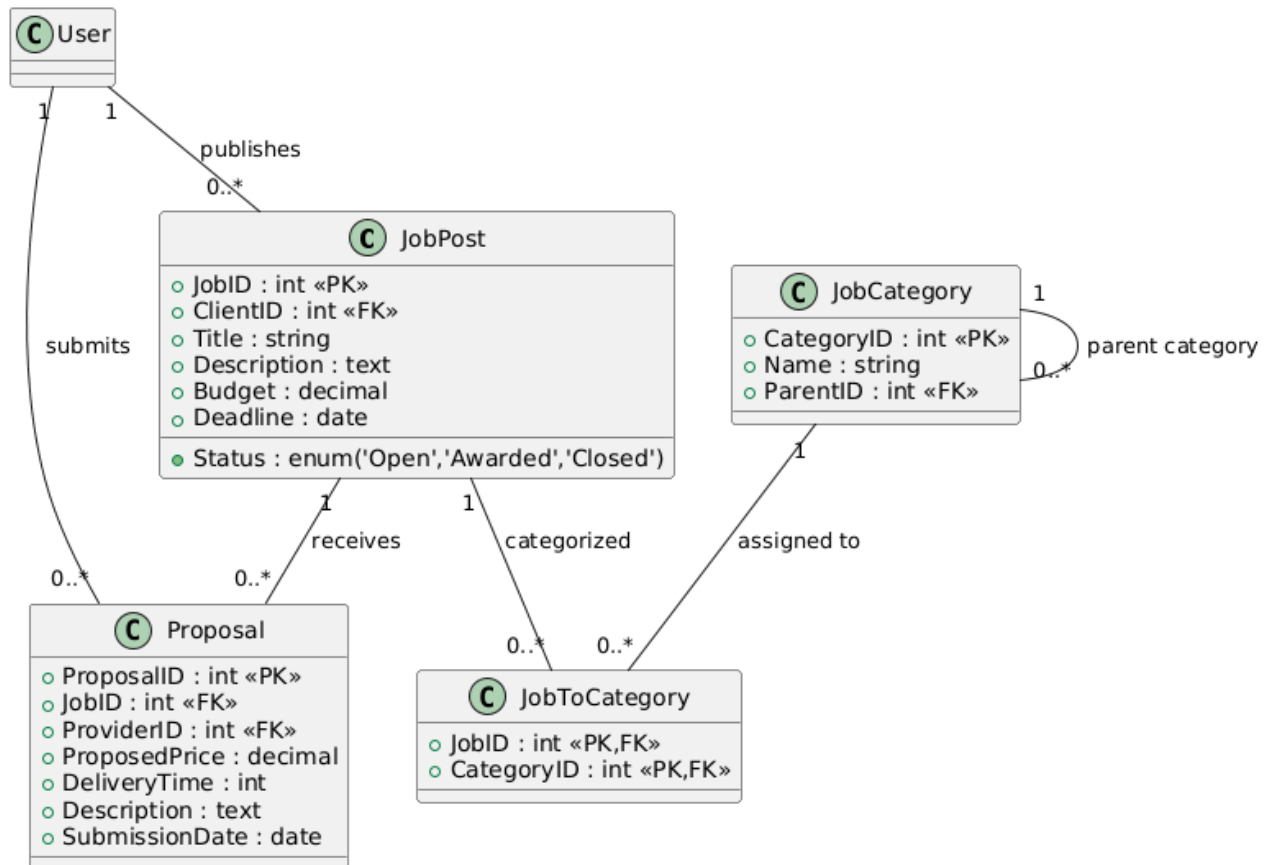
6.1



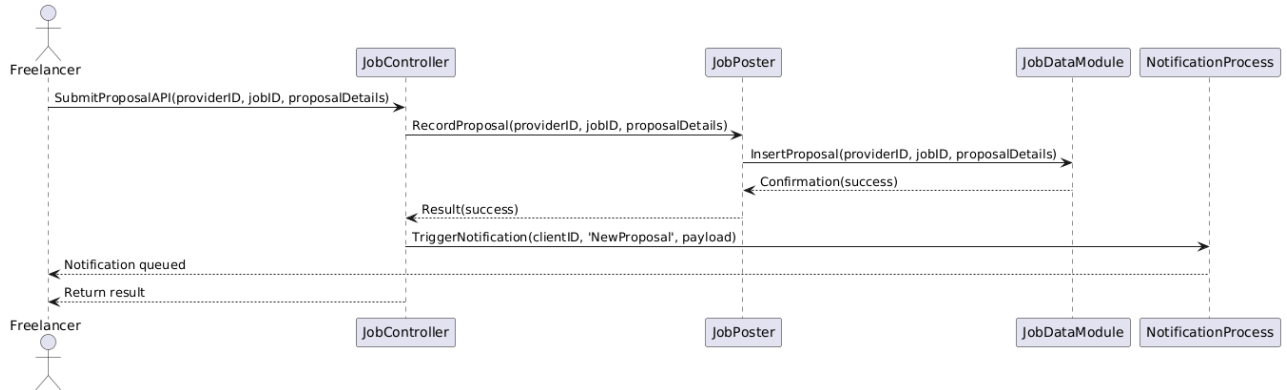
6.2.1



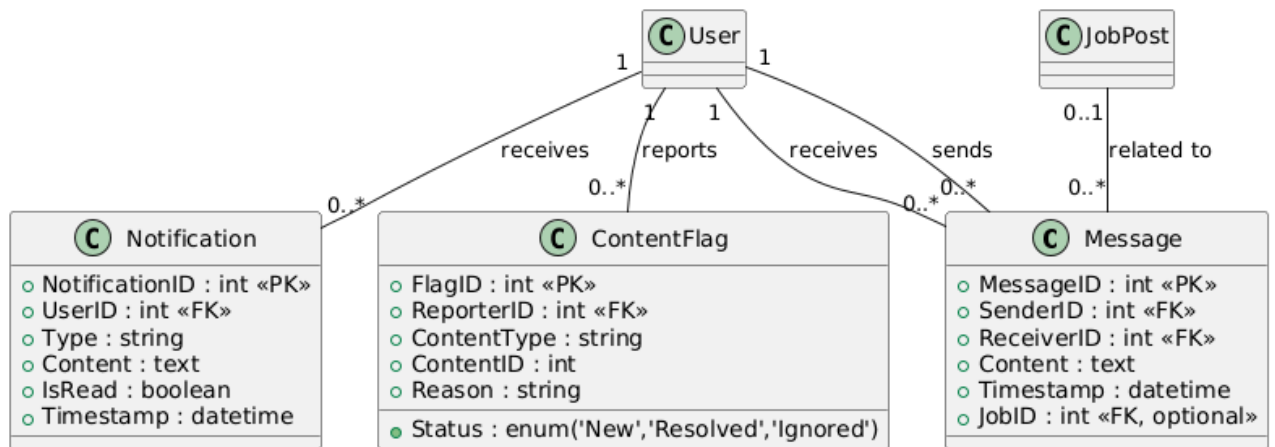
6.2.2



6.2



6.2.3



A6. Document evolution

The was finished on 17.Nov.2025. The evolution of the project can be observed on the 4th page.

A7. Conclusions regarding the activity

The activity has been very challenging but we successfully completed it, managing to creating a well structured document, well done diagrams and correct information.